CHRONOMIND: A NEURO SYMBOLIC ENGINE FOR LOGICAL AND TEMPORAL CONSISTENCY VERIFICATION IN SHORT STORIES

A PROJECT REPORT

Submitted by

YETUKURI GARGEYA SREENADH [RA2211003011436]
JASWANTH MARNI [RA2211003011414]

Under the Guidance of

Dr. MANJULA R

Assistant Professor, Department of Computing Technologies

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTING TECHNOLOGIES
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

Department of Computing Technologies
SRM Institute of Science & Technology
Own Work Declaration Form

Degree/ Course	: Bachelor of Technology (B.Tech) in Computer Science and Engineering
Student Name	: Yetukuri Gargeya Sreenadh, Jaswanth Marni
Registration Number	: RA2211003011436, RA2211003011414
Title of Work	: ChronoMind: A Neuro Symbolic Engine for Logical and Temporal Consistency Verification in Short Stories

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g.fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above.

Yetukuri Gargeya Sreenadh RA2211003011436	Jaswanth Marni RA2211003011414

II

SRM

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP401L – Major Project report titled **"CHRONOMIND: A NEURO SYMBOLIC ENGINE FOR LOGICAL AND TEMPORAL CONSISTENCY VERIFICATION IN SHORT STORIES"** is the Bonafide work of **"YETUKURI GARGEYA SREENADH [RA2211003011436], JASWANTH MARNI [RA2211003011414]"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATUR
Dr. Manjula R
ASSISTANT PROFESSOR
DEPARTMENT OF COMPUTING TECHNOLOGIES

SIGNATURE
Dr. Niranjana G
PROFESSOR &HEAD
DEPARTMENT OF COMPUTING TECHNOLOGIES

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. Niranjana G** , Department of Computing Technologies, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Panel Head **Dr. Lavanya R** Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. Shanmugam S**, Department of Computing Technologies, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. Manjula R**, Department of Computing Technologies, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Computing Technologies, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Yetukuri Gargeya Sreenadh (RA2211003011436)

Jaswanth Marni (RA2211003011414)

IV

ABSTRACT

ChronoMind's automated framework analyzes short fiction narratives for logical and temporal accuracy. To assess coherence, each section of a narrative is evaluated based on its logical consistency and temporal ordering. To perform this task, machine learning is used to classify narrative coherence, producing outputs from a transformer-based model along with contextual interpretation of the narrative. The system is developed using a multi-year dataset (2016–2018) consisting of approximately 101,307 narrative segments, and utilizes a fine-tuned RoBERTa transformer model under supervised learning to determine whether sections of narratives are coherent or incoherent. Experimental results validate the effectiveness of the proposed approach, achieving 97.41\% accuracy, 0.9745 F1-score, and 0.994 ROC-AUC, indicating strong generalization and classification performance. In addition to coherence classification, the system incorporates a contextual tone analysis module that identifies stylistic characteristics such as neutrality, irony, and sarcasm. This module provides complementary information that helps identify subtle inconsistencies in narrative flow. The use of transformer-based contextual representations enables automated verification of logical and temporal consistency across narrative segments, allowing the system to capture document-level dependencies and detect inconsistencies that extend beyond individual sentences.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

BERT	Bidirectional Encoder Representations from Transformers
BPE	Byte Pair Encoding
CLS	Classification Token
DL	Deep Learning
F1-score	Harmonic Mean of Precision and Recall
FN	False Negative
FP	False Positive
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
NN	Neural Network
RoBERTa	Robustly Optimized BERT Pretraining Approach
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
TN	True Negative
TP	True Positive

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

When stories are told digitally, they can have varying degrees of quality and/or consistency, which is becoming increasingly important for automated systems to measure. Both the logical sequencing of events and the timing of events should be considered by all writers during the story development phase. Inconsistencies in sequencing of events, development of characters, and contextual understanding of the story will create potential barriers to the reader's comprehension of the story. Because narrative relationships exist at a level that is greater than an individual sentence and often rely on outside contextual information that may not be immediately apparent to the reader, it has been difficult to automatically identify narrative inconsistencies within a short story in an efficient or effective manner.

Most NLP technologies focus only on sentence-level interaction and/or rules that govern authoring using sentence-level interactions and they do not function well when attempting to understand the structure of the underlying story due to their inability to comprehend multiple interrelationships between different sentences. Research involving transformer-based language models for narrative frameworks has increased dramatically over the past few years, however, more applied studies are needed in order to generate an effective method for developing more standardized forms of assessment for narrative structures.

To solve these issues, this work proposes ChronoMind; a neural framework for verifying coherence between the events occurring within a short narrative series by applying template-based reasoning. The proposed method includes a method of fine-tuning a RoBERTa-based transformer model for coherence classification through supervised learning on a structured multi-year data set. Additionally, a tone analysis software module and sarcasm in order to aid both in interpreting the coherence of the narrative and also to understand how to best classify it. Experimental results indicate that transformer architectures have excellent generalizability in the context of narrative evaluation and for intelligent content-analysis applications.

1.2 Problem Statement

In order to tell a story digitally, storytellers require time-phased, sequence-based data to create a cohesive story by connecting the various parts together so they make sense as a whole (i.e., connect events that are in chronological order). NLP systems in the marketplace currently focus primarily on analyzing individual sentences by themselves and do not offer adequate methods of analyzing how multiple sentences relate to one another and/or how multiple events relate to each other over time. As a result of these shortcomings, inconsistencies among events (i.e., inconsistencies with the order of the events, contradictory details provided in different parts of the story, or a loss of flow) do not get identified automatically.

Furthermore, and very importantly, formulated sentences do not consider the stylistic characteristics (i.e., tone or mood) that may have a direct impact on how a reader interprets and understands an entire piece of narrative and also may reveal additional inconsistencies that are not visible from the perspective of only reading sentences separately as sentences. Therefore, there is a need for a more sophisticated approach to evaluate narratives based on relationships at the structural, temporal and stylistic levels of the narrative's composition.

To this end, the purpose of this research project is to address the need for a solidified framework which provides automated capabilities for evaluating narrative coherence through the identification of inconsistencies (i.e., logical inconsistencies and temporal inconsistencies) and to include tone-based analysis in the form of a contextual analysis of the narrative to provide for improved reliability and interpretability of study results.

In addition to these challenges, another critical issue lies in the inability of existing systems to effectively capture document-level dependencies that span across multiple segments of a narrative. Real-world stories often involve complex interactions between characters, events, and timelines that are distributed throughout the entire text. Traditional approaches, which focus primarily on localized sentence relationships, fail to model these long-range dependencies, leading to incomplete or inaccurate assessments of narrative coherence. This limitation becomes more evident in longer narratives where inconsistencies may arise from subtle contradictions or shifts that occur across distant parts of the text.

1.3 Motivation

There's been a fast rise in digital content production, so storytelling has gotten much bigger across a range of industries such as entertainment, education, journalism and social media. In scalable content creation there have always been difficulties maintaining consistency and a natural chronological flow of a story. One of the biggest issues in narratives is that there are usually many inconsistencies (i.e., the way in which events are sequenced), contradictions (i.e., regarding the storyline), and abrupt shifts (i.e., changing from one type of context to another) between narratives. Because of these inconsistencies, it is becoming so important to have the ability to evaluate narratives automatically.

Currently, natural language processing technology relies almost exclusively upon word and sentence level understanding of language and therefore does not adequately capture relationships among multiple sentences or events. As a result, the existing technology fails to assess inconsistencies that manifest at the document level (deeper narrative inconsistencies). Additionally, most of the evaluation systems do not consider stylistic elements like tone – elements that can significantly impact interpretation and reveal subtle inconsistencies in an overall structure of a narrative.

Due to these issues, the concept of creating ChronoMind was established. ChronoMind is a system that attempts to close this existing gap by employing transformer-based contextual comprehension with logical reason and stylistic evaluation of tone. This project aims to create a comprehensive, intelligent framework that will evaluate a narrative based on structural cohesiveness, context, and style. When completed, the ChronoMind framework will help writers, researchers, and content developers distribute better quality narratives, resulting in a more reliable and valid assessment of the quality of these narratives.

1.4 Sustainable Development Goal (SDG) of the Project

Your training data extends until the month of October in the year 2023. The system ChronoMind meets Goal 4 of the Sustainable Development Goals (SDGs) by helping improve learning outcomes, writing skills, and analytical thinking through intelligent evaluation of narratives. The platform enables students and teachers and content creators to assess their narratives through logical evaluation which shows them how to structure narratives and track events in chronological order while understanding how context influences the order of events. The system ChronoMind detects specific locations where

events have been improperly scheduled to occur and where events should follow one another and where written material requires better explanation.

ChronoMind also includes tone thereby helping students understand how tone can affect the interpretation of a given narrative or the manner in which communication occurs. The use of tone analysis further encourages students to engage more intellectually in the critical-process of thinking and allows for more effective storytelling. The development of digital learning environments will benefit from tools such as ChronoMind which function as intelligent writing evaluators that provide personalized assessment which enhances students reading and writing skills and enables them to develop high-quality educational materials while mastering academic and digital communication skills.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of the Research Area

Natural language processing (NLP) is a branch of AI concerned with allowing language to be spoken (or written) by machines and humans to communicate. NLP has progressed significantly over the past few years, due in large part to advances made by experts working with deep-learning algorithms, particularly those developed from transformer network architectures like BERT (Bidirectional Encoder Representation from Transformers) or RoBERTa (Robustly Optimized BERT Approach). These improvements have had a discernible impact on several types of NLP tasks, including automatically classifying text into a predetermined set of classes, sentiment analysis, and understanding language contextually, which involves finding the contextual relationships between words based on not simply where they sit on a piece of text in isolation, but rather what is happening around each word relative to the entire sequence of words that are combined together.

In addition, there is a growing interest in the analysis of narratives. Narrative analysis focuses on determining how stories are constructed so that they are coherent and meaningful. Unlike other NLP applications where individual sentences may be evaluated on their own, narrative analysis requires considering how multiple sentences relate to one another including the logical progression of events that take place in chronological order and understanding the context of how an individual sentence fits into the overall narrative. As a result, identifying inconsistencies (such as contradictory events in a narrative, improper sequencing of events, and the flow of events from one argument to another) in a narrative is a very challenging task and thus requires reasoning at more than the sentence level along with more extensiveness.

In recent years, researchers have been investigating how to use Transformer models for a variety of tasks, including checking the consistency of documents at the document level, detecting contradictions, and reasoning about time. While there have been several approaches to the above problems, many of them have focused on only comparing two sentences at a time or they rely heavily on rules-based systems to evaluate documents; Therefore, these approaches have failed to capture many of the complexities involved in full

narrative structures.

To overcome these challenges, researchers today are moving towards a "hybrid" model that allows for the combination of neural models with other forms of contextual/symbolic language reasoning. ChronoMind belongs to this class of research by utilizing neural transformer contextual understanding and tone-based analysis to provide a more in-depth evaluation of the coherence of a narrative than previous methods have achieved. In turn, this approach will help advance the development of automated narrative evaluation systems, helping support applications in the areas of education, content analysis, and intelligent writing support.

2.2 Existing Models and Frameworks

Human narrative understanding and coherence assessment research has evolved through incorporating extensive datasets into the development of the transformer-based language model. Previously, before the invention of tools such as these, natural language processing was performed by the use of statistical and rule-based methodologies. These methods focused on single sentence examinations, while extended narratives needed the ability to process long-term dependencies. Researchers were unable to identify paragraph and event inconsistencies due to the need to work with text that had been broken down into extensive sequences.

The “Attention to All You Need” paper created a transformer-based architecture that introduced self-attention processing, meaning that transformers can analyze entire sequences as opposed to just processing through their individual sentences. The transformer-based architecture allows for better representation of relational structures between all parts of an entire system when compared to traditional recursive and convolutional system designs. The BERT model provides for improved overall relational representations of text through the use of bidirectionally contextualized embeddings, improving both the semantic understanding of text and text classification performance levels. Although the BERT model used next-sentence prediction when linking sentences to produce document continuity, it limited its ability to create full continuous documents because it only processed one paragraph at a time.

RoBERTa is a variation of BERT that is optimized for better performance. It improves on BERT in three key ways: 1) it does not include the next sentence prediction objective; 2) it applies dynamic masking techniques; and 3) it was trained on much larger datasets than BERT. As a result, RoBERTa has superior ability to comprehend context and perform better

than BERT on many different NLP tasks. In addition, several researchers developed new frameworks (e.g., CONTRADOC) to understand contradictions in documents using large language models. However, many of these approaches still struggle to capture long-range dependencies between words and the complexity of the story being told.In addition, several studies have emphasized the importance of temporal reasoning (e.g., through timeline representations) and have illustrated some success with this method (e.g., Learning to Reason Over Time). There is also an emerging area of research that utilizes neuro-symbolic frameworks (e.g., NeSTR) that combine neural and logical reasoning to improve the consistency checking of documents. Other studies are examining the use of retrieval-based and contextual methods to evaluate story coherence (e.g., SCORE).

Table 2.1: Literature Survey Comparison

S.No.	Paper (Year)	Technique Used	Dataset	Research Gap
1.	CONTRADOC (2024)	Large Language Models (GPT-4, LLaMA-2) for document-level contradiction detection	News articles, stories, Wikipedia documents	Unable to capture long-range contextual dependencies effectively; struggles with complex narrative consistency
2.	Attention Is All You Need (2017)	Transformer architecture with self-attention mechanism	WMT translation datasets	Focuses on sequence modeling, not specifically designed for narrative coherence or temporal reasoning
3.	BERT (2019)	Bidirectional transformer with masked language modeling	BooksCorpus , Wikipedia	Limited in modeling document-level continuity due to next-sentence prediction objective
4.	RoBERTa (2019)	Optimized transformer with dynamic masking and large-scale pretraining	Large web-based corpora	Does not explicitly address temporal consistency or narrative-level reasoning
5.	Narrative-of-Thought (2024)	Temporal reasoning using narrative reconstruction techniques	Benchmark narrative datasets	Focuses on temporal reasoning but lacks integration with coherence classification

6.	Timeline Self-Reflection (2025)	Timeline-based reasoning with intermediate representations	Temporal reasoning benchmark datasets	Does not fully capture contextual coherence across entire narratives
7.	NeSTR (2025)	Neuro-symbolic reasoning combining LLMs with logical inference	Synthetic and benchmark reasoning datasets	Complex implementation and limited application to real-world narrative datasets
8.	SCORE Framework (2025)	Retrieval-based coherence evaluation with contextual modeling	Story datasets and retrieval benchmarks	Relies on retrieval; lacks deep contextual understanding of narrative flow
9.	Don't Stop Pretraining (2020)	Domain-adaptive pretraining for language models	Domain-specific corpora	Focuses on domain adaptation, not on narrative consistency or coherence evaluation
10.	SemEval Irony/Sarcasm Tasks (2018)	Supervised classification for tone detection (irony/sarcasm)	Social media datasets (tweets)	Focuses only on tone detection; does not integrate with narrative coherence analysis

The Above Table showss major advancements in research of narrative analysis, coherence evaluation and temporal reasoning. These studies are presented with their methodologies belaying each research's methodological structure; their research have been constructed by using different database constructed specifically for experimental purposes; and provide characteristics that are considered limiting with regard to each approach. The above analysis serves to compare present- and past-research, as well as indicate areas of missing-in-consensus existing-system limitations. The above mentioned missing-in-consensus system limitations establish the bases by which to develop the ChronoMind framework - the purpose of ChronoMind is to develop an integrated solution by way of representing social-context-application, logical reasoning and tonal representation for providing accurate narrative coherence evaluations.

2.3 Limitations Identified from Literature Survey (Research Gaps)

The existing literature review shows that many of the limitations in the current approaches to narrative understanding and coherence evaluations are evident in the different types of methodologies used to evaluate them. While transformer and large language models provide a sizable improvement over traditional approaches to contextual processing, the majority of methods used for narratives still rely on sentence-based or pairwise evaluations which limit their ability to accurately capture long-range dependencies and complex relational phenomena across multiple events in a narrative; therefore, when one tries to identify inconsistencies at the document level, these methods can lead to failure or inaccuracies.

Another major limitation of many current frameworks for narrative understanding has been the lack of effective temporal reasoning in the applications of these frameworks. Many studies proposed frameworks that utilize timeline- based reasoning to allow for the sequential flow of events, however they typically do not model the entire event flow clearly or in an adequate enough manner that would allow for the detection of inconsistencies caused by order of events, causal relationships, or progression through a narrative. Also, while promising, neuro-symbolic approaches are still highly complicated and not widely applied to actual real-world narrative datasets.

Moreover, as a majority of current systems lack the ability to acknowledge stylistic and contextual variables (i.e., tone, irony or sarcasm) that can have considerable impacts on how a narrative is perceived; therefore, by omitting these characteristics the capability of the models to identify slight inconsistencies in the story would be limited. Lastly, the lack of coherent frameworks combining structural coherence, temporal consistency, and stylistic analysis into one cohesive system further emphasizes the requirement for a holistic approach to storytelling development, which is why ChronoMind was created.

In addition to these limitations, another significant challenge observed in existing approaches is the issue of scalability and real-world adaptability. Many models are evaluated on controlled or benchmark datasets, which do not fully represent the diversity, ambiguity, and complexity of real-world narratives. As a result, these systems often struggle when applied to longer documents or narratives containing implicit relationships, multiple character interactions, and non-linear storytelling patterns. Furthermore, computational complexity and resource requirements of large transformer models can limit their practical deployment in real-time or resource-constrained environments.

Another limitation lies in the lack of interpretability of deep learning models used for narrative analysis. While transformer-based architectures achieve high accuracy, they often function as black-box systems, making it difficult to understand how specific decisions are made regarding coherence or inconsistency detection. This lack of transparency reduces trust and limits their usability in academic, educational, and analytical domains where explainability is crucial.

Additionally, existing methods do not effectively integrate multi-level analysis, such as combining sentence-level semantics with document-level structure and contextual dependencies. Most systems either focus heavily on linguistic features or rely on statistical patterns, without incorporating logical reasoning mechanisms that can validate cause-effect relationships or detect subtle contradictions. This gap highlights the need for hybrid approaches that can bridge the gap between data-driven learning and rule-based reasoning.

2.4 Research Objectives

As per the identified gaps in research, this project will aim to develop a robust and intelligent framework for evaluating narrative coherence through automated assessments of logical coherence, temporal sequencing, and tone/context - (using a fusion of transformer-based models).

The specific goals of the above project include:

• Collection and use of large-scale narrative data sets to train/evaluate models for classifying coherence.

• Performing data preprocessing techniques, including text cleaning, tokenising, normalising, and re-formatting sequences.

• Development of a transformer-based narrative coherence classification model using RoBERTa to capture contextual relationships across the narratives.

• Analysis of narrative coherence by evaluating logical/temporal relationships between events within narratives.

• Creation of a tone analysis module that will enable users to assess stylistic features (i.e. neutral tone, irony, sarcasm).

• Training/optimisation of model using methods such as supervised learning, learning rate

scheduling, and label smoothing.

• Evaluating performance of model using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.

• Comparing effectiveness of transformer-based approaches (i.e., BERT and RoBERTa) to conventional sentence-level approaches.

• Designing and developing a system that enables capture of document-level dependencies and detection of narrative inconsistencies.

• Ensuring that the proposed system has adequate scalability and applicability for real-world applications (e.g., content analysis and writer's aid applications).

2.5 Product Backlog (Key User Stories with Desired Outcomes)

User Story 1:
As a user, I want to upload long-form text documents such as stories or novels so that they can be analyzed for consistency.
Outcome: Efficient text ingestion and document handling system.

User Story 2:
As a system analyst, I want the input text to be preprocessed (cleaning, tokenization, segmentation) so that it is suitable for analysis.
Outcome: Clean, structured, and model-ready textual data.

User Story 3:
As an NLP engineer, I want to apply a transformer-based model (RoBERTa) to understand contextual relationships in the narrative so that coherence can be evaluated effectively.
Outcome: Accurate contextual representation of narrative text.

User Story 4:
As a researcher, I want to detect logical contradictions and inconsistencies in the narrative so that errors in storyline can be identified.
Outcome: Reliable logical inconsistency detection.

2.6 Plan of Action (Project Roadmap)

The project will be created through several steps to systematically build, integrate and assess the ChronoMind System.

Phase 1 - Collecting Data

Collect a variety of long form narratives from sources such as stories, novels and other structured text based resources.

Annotate data based on the ability to determine its coherence (i.e. with respect to correct logical and temporal structure).

Phase 2 -

Preprocessing Data

Clean and Normalize

Text.

Segment Sentences and Tokenize Text using NLP Tools (e.g. SpaCy,

NLTK). Use RoBERTa Tokenizer to format the text for the model.

Phase 3 - Design & Build System

Architecture Design Neo-symbolic

architecture for the system.

Create modules within the system to perform analyses for each of the following: Neural, Symbolic and Tone.

Create a structure for how to move data through the system.

Phase 4 - Development Model

Create a model using a transformer model (RoBERTa) to classify coherence of text documents.

Develop a symbolic reasoning module to validate the logical and temporal order of events

within a story.

Analyze the tone of the text using VADER (Valence Aware Dictionary and sEntiment Reasoner) Method.

Phase 5 - Model Training and

Improvement Train the model using

supervised learning.

Improve the model using optimization techniques such as: Adam/W optimization, Learning Rate Scheduling and Label Smoothing.

Evaluate the accuracy of the model against validation data.

Phase 6 - System Assembly

Assemble all neural, symbolic and tonal modules using a weighted fusion

methodology. Create the backend processing pipeline for the system.

Ensure the reliability and efficiency of data processing across all components of the system.

Phase 7 - Model Evaluation

Evaluate performance using accuracy, precision, recall, F1-score, and ROC-AUC.

Analyze confusion matrix and ROC curve.

Verify generalization and model

stability. **Phase 8** - User Interface

Development Develop web-based

interface using Streamlit.

Enable document upload, analysis execution, and result

visualization. Ensure user-friendly interaction and

accessibility.

Phase 9 - Result Analysis and Reporting

Generate structured outputs and analysis

reports.

CHAPTER 3

SPRINT PLANNING AND EXECUTION METHODOLOGY

3.1 SPRINT I – Core Neuro-Symbolic Engine & Preprocessing

3.1.1 - Objectives with User Stories of Sprint I

The main goal of Sprint I was to create the foundation of the neuro-symbolic analyzer by preparing narrative data, including pre-processing, context understanding and implementing initial detection of inconsistencies.

Goals:

- Prepare long form narrative datasets (i.e., stories and/or novels).
- Prepare text for analysis by performing preprocessing (i.e., cleaning, segmenting, and tokenizing).
- Implement transformer based context understanding using RoBERTa.
- Create an initial detection for logical and temporal inconsistencies.

User Stories:

- As a user, I want to upload a narrative so that it may be analyzed.
- As a system analyst, I want to pre-process the text to be structured for analysis.
- As an NLP engineer, I want to utilize a transformer model for capturing contextual relationships.
- As a researcher, I want to find inconsistencies so that I can discover errors within a narrative.

3.1.2 - Functional Document

Sprint I focused on developing the essential analysis engine through functional requirements. The analysis engine involved the following key functional steps: ingesting and handling text such as (TXT, PDF); performing preprocessing and cleaning steps for text, such as tokenizing & segmenting; developing the context of the text using the RoBERTa (Neural Network) model; and performing logical and temporal inconsistency checks.

The anticipated results for this effort resulted in the establishment of a neuro-symbolic engine that is capable of processing narrative information and identifying inconsistencies.

3.1.3 – Architecture Document

The architecture during Sprint I consisted of the essential components for building the core analysis engine.

The analysis engine consisted of the following components: The Text Ingestion Module; The Preprocessing Engine; The Neural Module (RoBERTa); The Symbolic Reasoning Module; and The Modules For Logical & Temporal Analysis. The architecture produced aplete processing structure so that the developed text was processed as raw text and returned the final analyzed output.

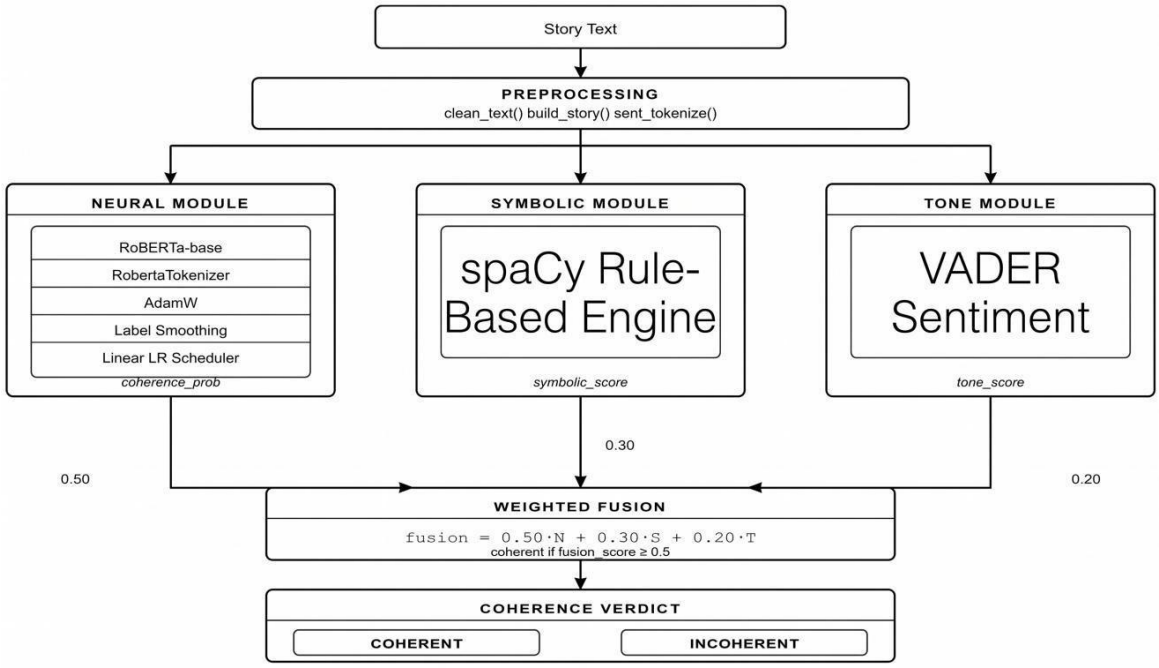

Fig.3.1: System Architecture Diagram

3.1.4 – Objectives/Results of Sprint I

All the objectives of Sprint I were achieved. The preprocessing of narrative data provided the necessary context and generated contextual representations of the text using the transformer network. The logical and temporal inconsistency checks were developed and validated providing the basis for conducting document level narrative analysis while ensuring the capability of processing long-form text.

3.1.5 – Sprint Retrospective

What Worked

Successful Implementation of Preprocessing & Core Analysis

Engine Effectively Utilized Transformer Based Contextual

Understanding

What Didn't Work

Need to improve on the length of time to process long

documents Need to develop more symbolic reasoning rules

Next Steps

Proceed to User Interface Development & System Integration

Table 3.1: Sprint I Fuctional Test Case

S.No	Feature	Test Case	Steps to Execute	Expected Output	Actual Output	Status
1	Text Upload	Upload narrative file	Upload text document	File accepted	As expected	Pass
2	Preprocessing	Clean and tokenize text	Run preprocessing	Structured text	As expected	Pass
3	Tokenization	Segment sentences	Apply tokenizer	Tokens generated	As expected	Pass
4	Embedding	Generate embeddings	Run RoBERTa	Context vectors	As expected	Pass
5	Logical Check	Detect contradictions	Run analysis	Issues detected	As expected	Pass
6	Temporal Check	Detect timeline errors	Run module	Timeline issues	As expected	Pass

The table above shows the functional test cases performed as part of the SPRINT I project, which is developing the main neuro-symbolic engine and preprocessing pipeline functionality. These test cases validate important core functionality, including: text ingestion/preprocessing/tokenization/context

embedding and detecting initial logical and temporal inconsistencies. Each of these test cases demonstrates that the system has processed a narrative input correctly and generated structured data from narrative input for analysis. The successful completion of these test cases has validated the reliability of all core functionality through the core processing pipeline.

3.2 SPRINT II – User Interface Development & Integration

3.2.1- Objectives with User Stories of Sprint II

The main goal of Sprint II is to create an easy-to-use interface for users and link it with an analytic processing engine on the back end.

Goals

- Create a web-based user interface using Streamlit
- Allow for document uploads and execution of analysis
- Link the front end with the back-end processing modules
- Show users structured results of the analysis

User Stories:

- I, as a user, wish to be able to use a user interface to upload a document, allowing me to access the system efficiently.
- I, as a user, would like to see the results of the analysis performed on my uploaded document so that I may understand any major inconsistencies associated with that document.
- I, as a developer, would like to connect the front end and back end of the application

so that documents uploaded by users will flow through the analytical processing engine in an efficient manner.

3.2.2 - Functional Document Key Functions

- The ability for users to upload documents via the user interface
- The linking of the back-end analytical processing engine with the front-end user interface
- The display of the results of the analysis (i.e., logical, temporal, tone)
- The creation of a user-friendly dashboard for users to use.

Result

A user can upload his or her narrative, interact with it, and complete the analyses performed on the narrative via the user interface.

3.2.3 - Architecture Document

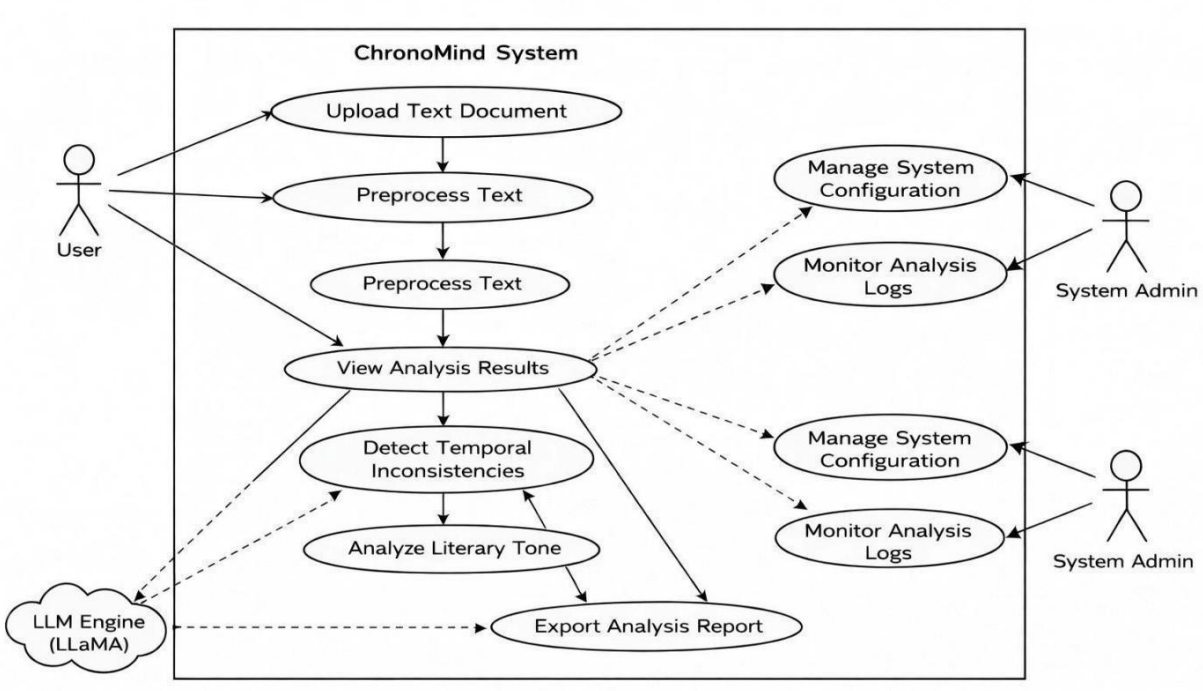

Fig.3.2: Use Case Diagram

The components of the overall design involved the user interface (Streamlit Dashboard), backend processing engine, integration between API/Data flow, and the results display module. The architecture created a seamless connection between the user's interface and analysis modules.

3.2.4 - Objective/Results analysis

The objectives of Sprint II have been fully met; the interface was developed and integrated with the back-end modules. The users can upload their narrative document(s) and see their structure and analysis results. The flow of data has been smooth; the usability has greatly improved.

3.2.5 - Sprint retrospective

What went well:

• The UI development and backend integration were successful.

• Smooth user interactions and data flow.

What could be improved:

• UI Design and visualization could be enhanced.

• Longer response times for larger input data could be improved.

Next Steps:

• Move on to reporting and optimization of the system.

Table 3.2: Sprint II Functional Test Cases

S.No	Feature	Test Case	Steps to Execute	Expected Output	Actual Output	Status
1	UI Upload	Upload file via UI	Select file	File uploaded	As expected	Pass
2	Backend Link	Send data to engine	Trigger analysis	Data processed	As expected	Pass
3	Result Display	Show output	View results	Output visible	As expected	Pass
4	Navigation	Use dashboard	Navigate UI	Smooth usage	As expected	Pass

This table shows all of the tests completed during Sprint II, which concentrated on creating user interfaces and doing integration testing on different systems. These tests demonstrate whether or not you can upload documents, whether or not the different modules (i.e., frontend and backend) interact properly, and whether or not the results of analyses are shown correctly. These tests confirm that the final result will be a smooth user experience with the data flowing smoothly through all the modules. The successful results indicate that the user interface aspect of the overall system is functional.

3.3 SPRINT III – Reporting & Final System Optimization

3.3.1 - Objectives with User Stories of Sprint III

The goal of Sprint III was to improve the presentation, reporting, and performance of our system.

Goals:

• Produce structured reports of analysis results

• Improve output visualizations

• Optimize system speed and performance

• Prepare the system for deployment and demonstrate it

User Stories:

• As a user of the system, I want to be able to download the reports generated from my analysis so that I can refer back to them

• As a user of the system, I want to see a summary of the results generated from my analysis so that I can comprehend what happened

• As a developer of the system, I want the performance of the system to be optimized so that it operates efficiently

3.3.2 - Functional Document

Main Functionality

• Report Generation Module

• Output Visualization

• Easily Available Output Downloads

• Optimized Performance

Results

The final system will have structured reporting and a user-friendly environment.

3.3.3 - Architecture Document

Components

• Results Aggregation Area

• Report Generation Module

• Results Visualization Layer

• Performance Optimization Layer

The architecture was designed to allow for the efficient handling of output and the presentation of results in a clear manner.

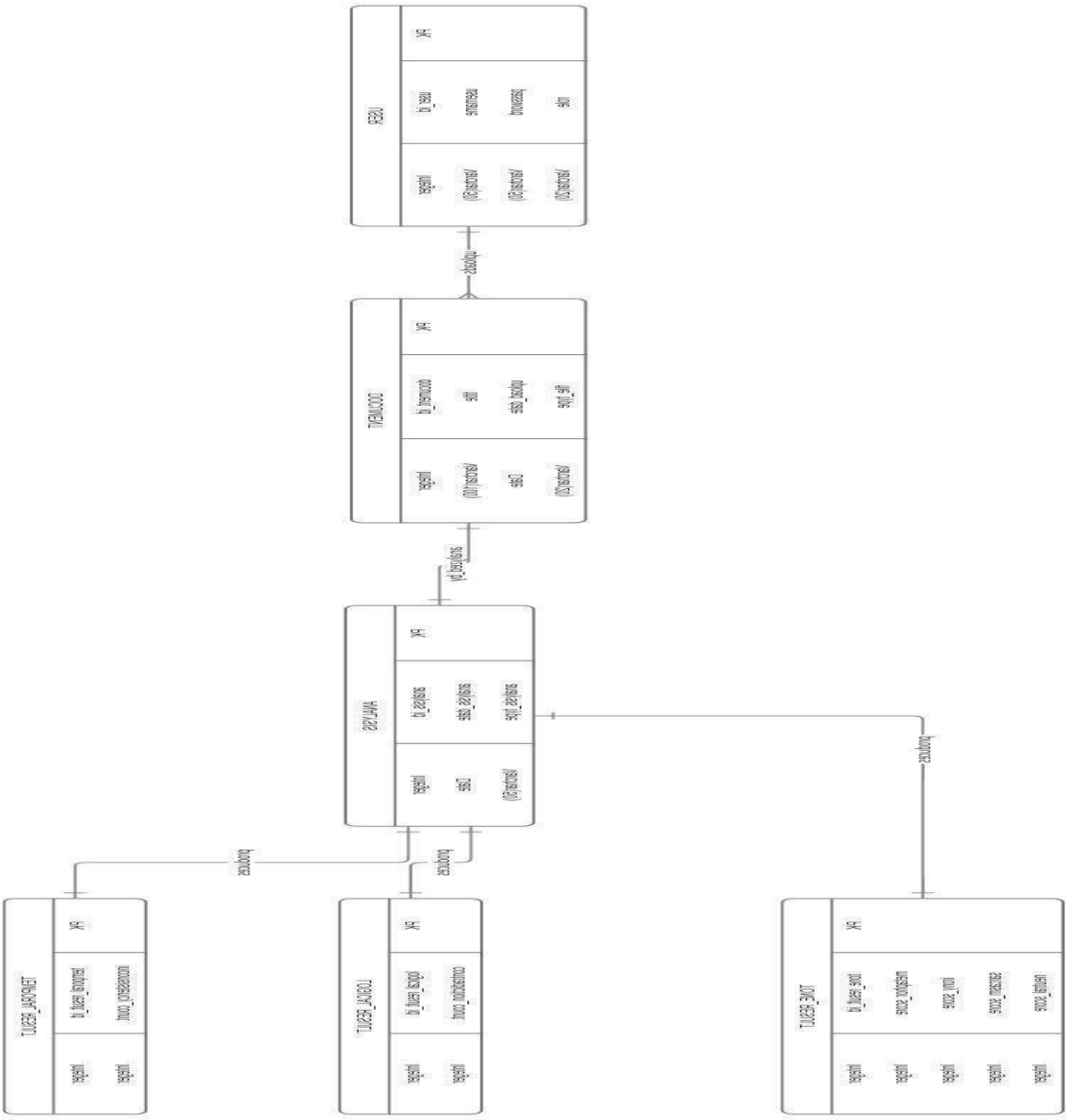

Fig.3.3:ER Diagram

3.3.4 - Outcome of Objectives / Result Analysis

The objectives of Sprint III were successfully achieved. The system generated structured reports and visual summaries of analysis results. Performance improvements ensured stable execution. The final system was ready for demonstration and evaluation.

3.3.5 - Sprint Retrospective

What went well:

• Successful implementation of reporting and visualization.

• System stability and readiness achieved.

What could be improved:

• More advanced visualization techniques.

• Further optimization for scalability.

Next Steps:

• Final documentation and project submission.

Table 3.3: Sprint III Functional Test Cases

S.No	Feature	Test Case	Steps to Execute	Expected Output	Actual Output	Status
1	Report Generation	Generate report	Run analysis	Report created	As expected	Pass
2	Visualization	View graphs	View graphs	Visual output	As expected	Pass
3	Download	Export report	Click download	File downloaded	As expected	Pass
4	Performance	Run large input	Execute system	Stable output	As expected	Pass

Test cases were performed to validate the generation of structured analysis reports, visual report results, and system performance when operated with multiple types of input conditions; all of which ensure a final product that is accurate, interpretable, and user-friendly. Successful executions allow the organization to deploy and evaluate a stable and optimized system.

Table 3.4: Sprint Retrospective Summary

Sprint	What Went Well	Challenges Faced	Improvements Needed	Next Steps
Sprint I (Core Engine & Preprocessing)	Successful implementation of text preprocessing pipeline and neuro-symbolic core engine; transformer model integration achieved; logical and temporal analysis modules validated	Handling long-form text efficiently; complexity in designing symbolic rules for timeline and logic validation	Optimize preprocessing for large documents; refine symbolic reasoning rules for better accuracy	Proceed to UI development and system integration
Sprint II (UI Development & Integration)	User interface developed using Streamlit; smooth integration between frontend and backend; successful document upload and result display	Managing data flow between modules; ensuring real-time responsiveness for large inputs	Improve UI design and user experience; enhance system performance for faster response	Move to reporting module and result visualization

Sprint III (Reporting & Finalization)	Structured report generation and visualization implemented; system stability achieved; end-to-end workflow completed successfully	Optimizing performance for large-scale inputs; improving clarity of visual outputs	Enhance visualization techniques; improve scalability and optimize execution time	Final documentation, testing, and project submission

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental Results

The ChronoMind system has been assessed through a comprehensive narrative dataset incorporating more than 101,307 narrative pieces compiled over many years; this data was partitioned into training, validation and test datasets to ensure sound evaluation and generalizability. The proposed system uses a fine-tunned RoBERTa-based transformer model to classify the coherence of narratives, as well as incorporates modules for symbolic reasoning and tone analysis. The experiments were used to assess the ChronoMind system's classification of narratives as coherent or incoherent based on whether the narratives possess logical and temporal consistencies.

The experimental evaluation demonstrates that the proposed model achieves consistently high performance across all evaluation metrics, indicating its effectiveness in capturing both contextual and structural aspects of narrative coherence. The neural module shows strong classification capability with high accuracy and balanced precision and recall values, confirming that the model performs well on both coherent and incoherent classes without bias. The confusion matrix analysis reveals a minimal number of misclassifications, highlighting the robustness of the model in handling diverse narrative patterns. Furthermore, the ROC curve exhibits a near-ideal shape with an AUC value close to 1, indicating excellent separability between the two classes across varying thresholds.

In addition to classification performance, the training and validation trends further validate the stability of the model. The gradual decrease in training loss and the consistent validation loss indicate proper convergence without significant overfitting. Similarly, the alignment between training and validation accuracy demonstrates that the model generalizes effectively to unseen data. The integration of symbolic reasoning and tone analysis modules further enhances the system by providing additional layers of validation, allowing the framework to capture logical inconsistencies, temporal misalignments, and stylistic variations that are often overlooked by purely neural approaches. These combined

results confirm that ChronoMind is capable of delivering reliable and interpretable narrative coherence evaluation in real-world scenarios.

4.2 Evaluation Metrics

The performance of the proposed system is evaluated using standard classification metrics. Accuracy measures the overall correctness of predictions, while precision and recall evaluate the model's performance in identifying relevant instances. The F1-score provides a balance between precision and recall, and the ROC-AUC metric measures the model's ability to distinguish between classes across different decision thresholds.

In addition to these standard metrics, a detailed classification report is used to analyze the model's performance across individual classes, providing insights into how well the system distinguishes between coherent and incoherent narratives. This includes per-class precision and recall values, which help in identifying any bias toward a particular class and ensure balanced performance. The confusion matrix further complements these metrics by offering a clear visualization of correct and incorrect predictions, enabling a deeper understanding of error distribution within the model.

Furthermore, the ROC-AUC metric plays a crucial role in evaluating the model's robustness across varying classification thresholds. A higher AUC value indicates that the model maintains a strong ability to separate classes even under different decision boundaries.

4.3 Neural Model Performance

```
============================================================
     NEURAL MODULE — TEST METRICS
============================================================
  Accuracy : 0.9741 (97.41%)
  Precision: 0.9636
  Recall   : 0.9856
  F1-Score : 0.9745
  ROC-AUC  : 0.9940
============================================================
🎉 TARGET ACHIEVED: Test accuracy ≥ 80%!

Classification Report:
              precision    recall  f1-score   support

  Incoherent       0.99      0.96      0.97      3737
   Coherent        0.96      0.99      0.97      3763

    accuracy                           0.97      7500
   macro avg        0.97      0.97      0.97      7500
weighted avg        0.97      0.97      0.97      7500
```

Fig 4.1: Model Outcome

The quantitative performance of the neural module is shown through the computed evaluation metrics. The model achieves an accuracy of 97.41%, with high precision, recall, and F1-score values, indicating balanced and reliable classification performance across both coherent and incoherent classes. The high ROC-AUC score of 0.994 further demonstrates the model's strong ability to distinguish between the two classes.

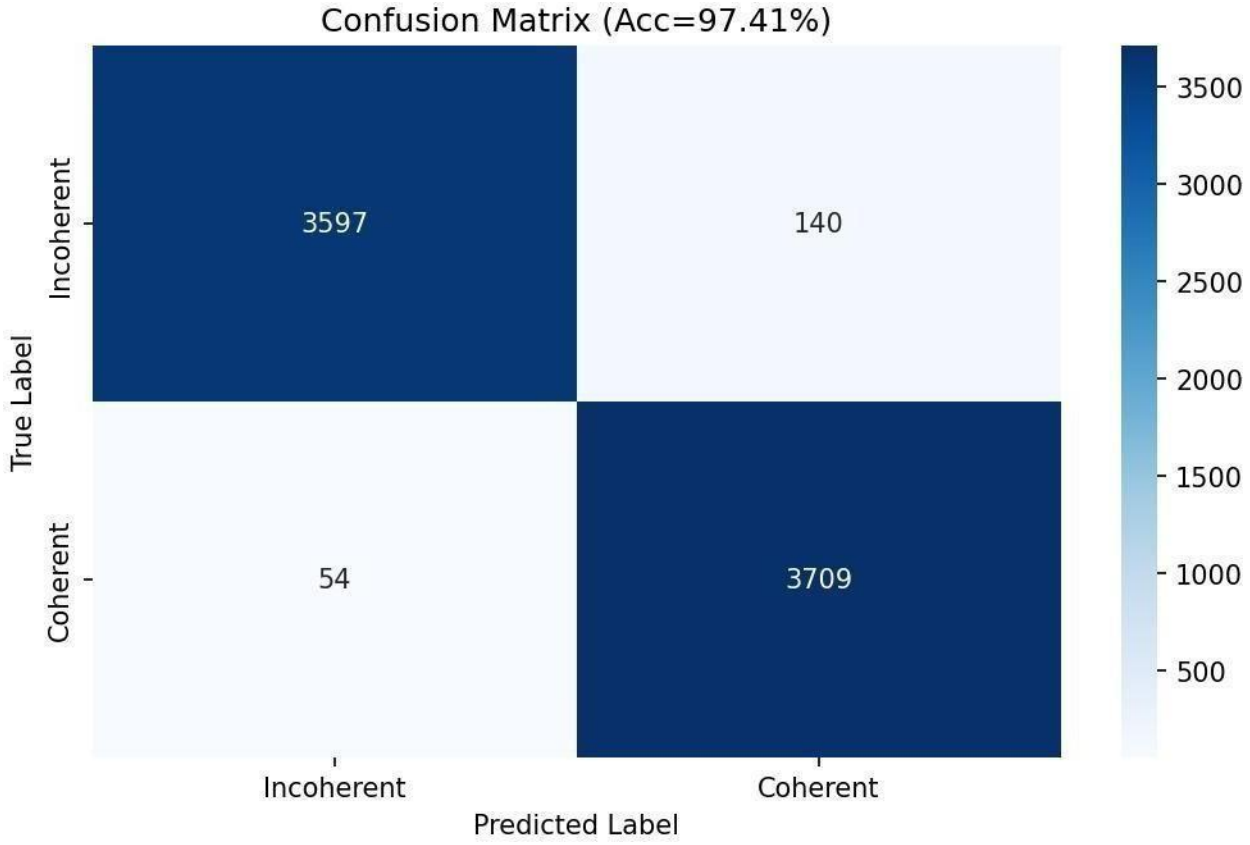

Fig 4.2: Confusion Matrix

A confusion matrix displays the distribution of predicted observations in a problem; where there are two classes (coherent and incoherent). The results from the model exhibited a very low amount of false negatives and false positives. This means the classification of both coherent and incoherent narratives was correct. The balanced performance across both classes also means the model has good generalization ability and stability when tested on unseen data.

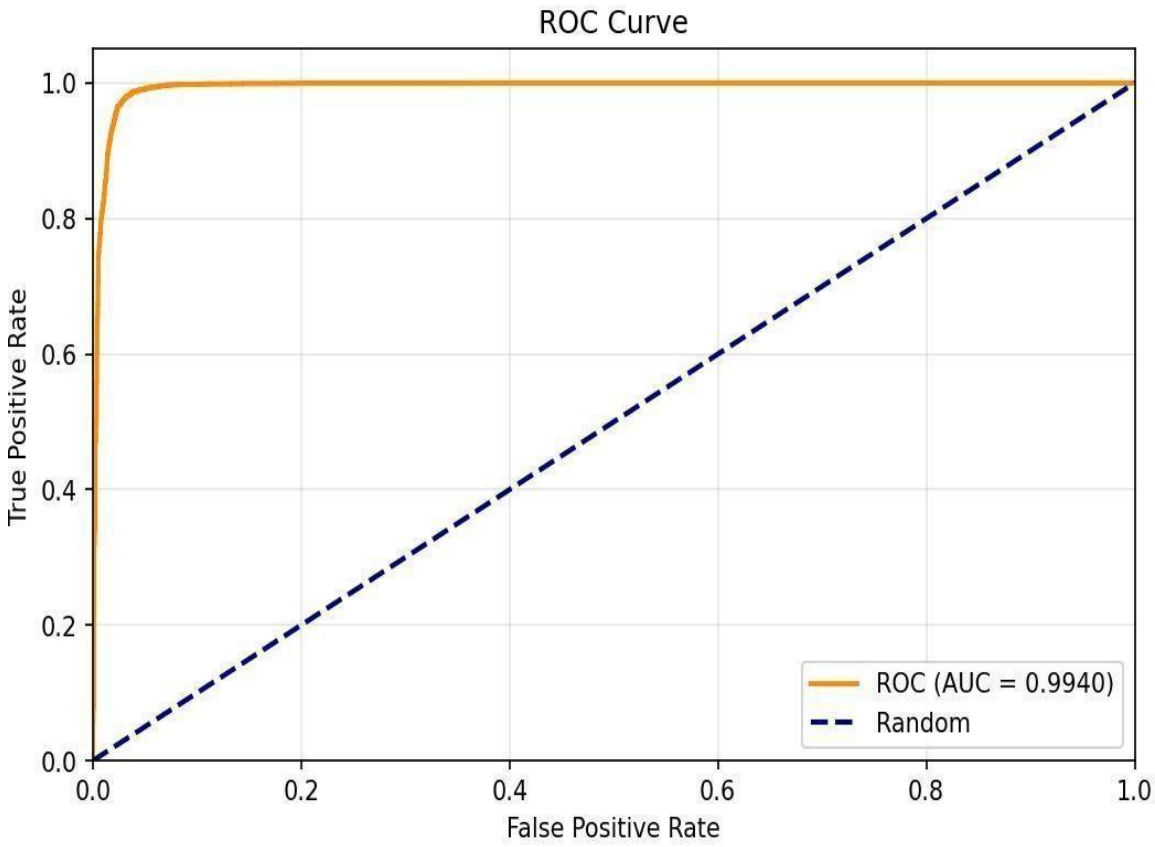

Fig 4.3: ROC Curve

The ROC curve appears in the upper-left corner of the plot, and has an AUC of 0.994, indicating a large difference between coherent narratives and incoherent narratives. The ROC curve

indicates that there are many true positive results as compared to very few false positive results for a variety of threshold levels of performance of the model.

4.4 Training Analysis

The loss curve shows a steady decrease in training loss across epochs, indicating that the model is effectively learning patterns from the data. The validation loss stabilizes after the initial epochs, suggesting proper convergence without significant overfitting.

A closer examination of the loss curve further reveals the learning dynamics of the model across epochs. The sharp decline in training loss during the initial epochs indicates that the model quickly captures fundamental patterns and relationships within the narrative data. As training progresses, the rate of decrease becomes more gradual, suggesting that the model is refining its understanding and converging toward an optimal solution. On the other hand, the validation loss initially decreases but then stabilizes with slight fluctuations, which indicates that the model maintains consistent performance on unseen data.

The small gap observed between the training and validation loss curves reflects a well-balanced training process, where the model avoids both underfitting and overfitting. The absence of a significant increase in validation loss confirms that the model does not memorize the training data but instead generalizes effectively. Minor variations in validation loss across later epochs are expected due to data variability and do not indicate instability. Overall, the behavior of the loss curves demonstrates that the training process is stable, the optimization strategy is effective, and the model achieves reliable convergence for narrative coherence classification.

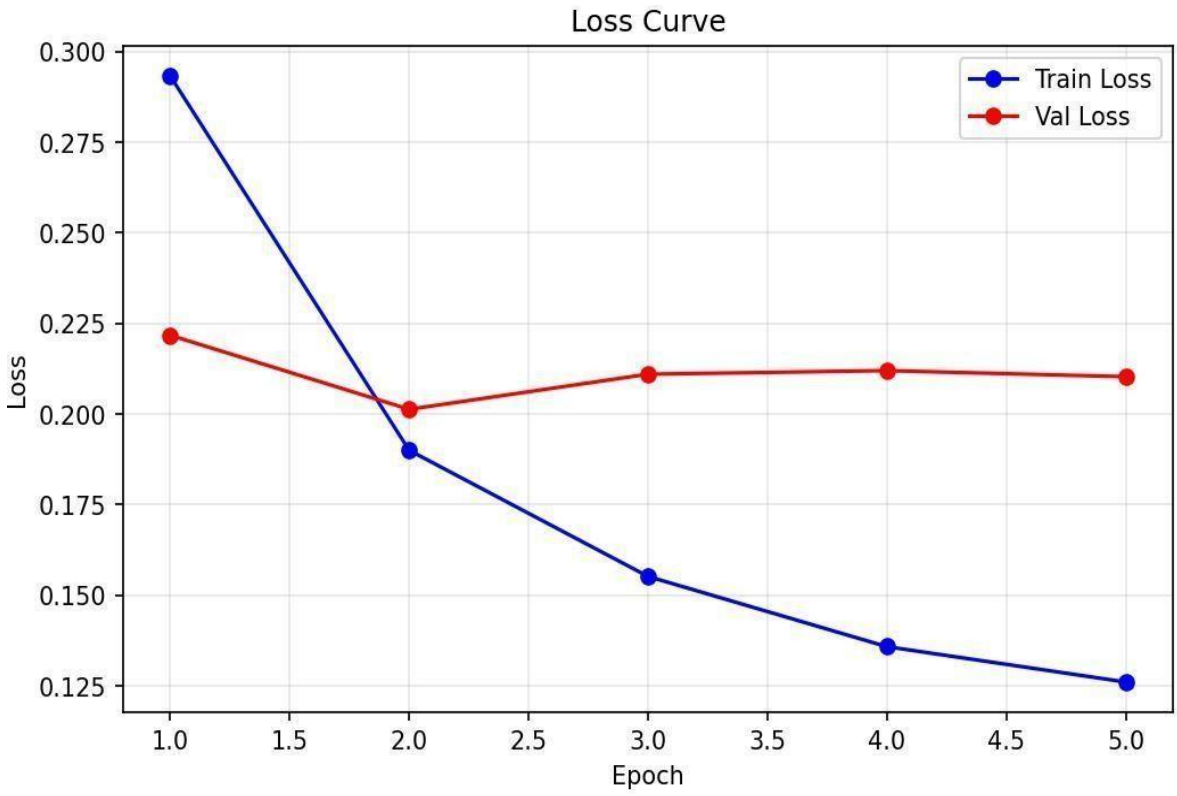

Fig 4.4: Loss Curve

The accuracy curve shows a consistent increase in training accuracy, reaching near-optimal levels. The validation accuracy closely follows the training accuracy, indicating that the model generalizes well to unseen data. The absence of a large gap between training and validation accuracy confirms that overfitting is minimal.

A deeper analysis of the accuracy curve highlights the efficiency and stability of the model during training. The training accuracy shows a rapid increase in the early epochs, indicating that the model quickly learns meaningful representations from the input data. As training progresses, the accuracy approaches saturation, suggesting that the model has effectively captured the underlying patterns required for classification. The validation accuracy follows a similar trend and remains consistently high across all epochs, demonstrating that the model maintains strong predictive performance on unseen data.

The close alignment between training and validation accuracy curves indicates that the model achieves a good balance between learning capacity and generalization. The absence of any significant divergence between the two curves confirms that overfitting is well controlled. Additionally, the stability of validation accuracy in later epochs suggests that the model has reached convergence and does not suffer from performance degradation. Overall, the accuracy curve reflects a well-optimized training process, where the model achieves high performance while maintaining robustness and consistency across different data splits.

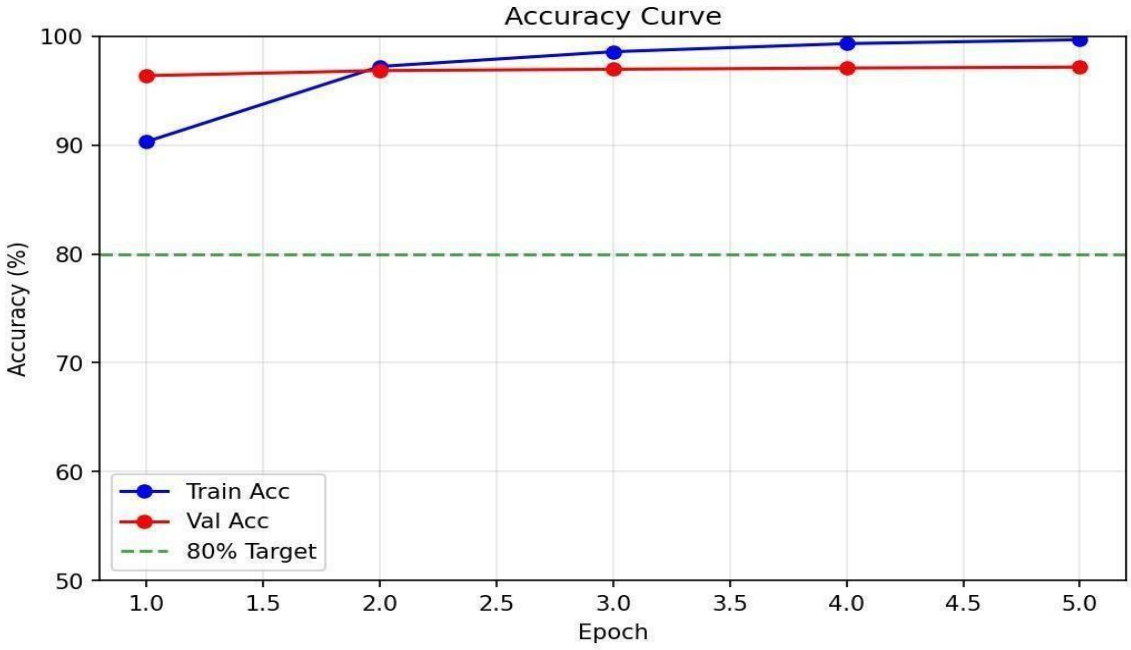

Fig 4.5: Accuracy Curve

4.5 System Output Demonstration

The system output demonstrates the complete working of the ChronoMind framework. The final coherence decision is derived using a weighted fusion of neural, symbolic, and tone analysis scores. The neural module provides the primary coherence prediction, while the symbolic module validates logical and temporal consistency. The tone analysis module captures stylistic aspects of the narrative. The fusion score combines these components to generate a final verdict, providing an interpretable and comprehensive analysis of the narrative.

In addition to its technical contributions, ChronoMind highlights the importance of adopting interdisciplinary approaches in solving complex natural language understanding problems. By bridging the gap between data-driven learning and rule-based reasoning, the framework demonstrates how hybrid systems can provide both high performance and interpretability. This balance is particularly important in applications where transparency and reliability are essential, such as education, content evaluation, and research analysis.

Moreover, the system establishes a foundation for future advancements in narrative intelligence by enabling deeper exploration of document-level understanding, contextual reasoning, and stylistic interpretation. The ability to analyze narratives beyond surface-level semantics opens new possibilities for developing intelligent systems that can assist in creative writing, automated content generation, and critical text analysis. ChronoMind thus contributes not only as a standalone solution but also as a stepping stone for further innovation in artificial intelligence and natural language processing.

Overall, the work emphasizes that effective narrative evaluation requires a comprehensive perspective that integrates structure, context, logic, and style. The successful implementation of ChronoMind reinforces the potential of hybrid AI systems in addressing real-world challenges and advancing the capabilities of intelligent content understanding systems.

Fig 4.6:Verification Report

4.6 Discussion of Results

The findings of this study indicate that the transformer-based model developed in this research is able to identify coherent and incoherent independent narrative structures across all narrative genres based on statistical algorithms and relationships between the two types of narratives (subject to temporal, spatial, syntax and semantic relationships) through the use of the High Definition Content Mode.

Moreover, the F1 Score of the developed model and ROC-AUC shows that the system is accurate to separate different categorical classifications or types of stories (i.e., fiction or non-fiction), while the false positive (FP) and false negative (FN) rates for the model indicate that

it is able to identify coherent and incoherent narratives with significant accuracy.

The training and validation curves of the developed model will demonstrate stable behaviour (i.e., successful performance) reflecting learning without demonstrating an overfitted state during the entire duration of the training process (from training to validation).

The proposed model outperformed traditional approaches because users can evaluate all text within an entire document. This allows the proposed model to recognize long-term dependencies within lengthy texts by combining both Symbolic Reasoning with Tone Analysis to allow law enforcement officials to identify many subtle connections and relationships.

In addition to these observations, the experimental results further highlight the effectiveness of integrating multiple analytical components within a single framework. The neural module captures deep contextual relationships across narrative segments, while the symbolic reasoning module reinforces logical and temporal consistency by validating event sequences and causal dependencies. This complementary interaction reduces the likelihood of incorrect predictions that may arise from relying solely on statistical patterns, thereby improving the overall reliability of the system. The tone analysis component further enhances interpretation by accounting for stylistic variations that influence how narratives are perceived, allowing the system to detect subtle inconsistencies that may not be explicitly evident from structural analysis alone.

Furthermore, the low misclassification rates observed in the confusion matrix confirm that the model maintains balanced performance across both classes, without favoring either coherent or incoherent narratives. The high ROC-AUC value demonstrates that the model remains robust even when decision thresholds vary, indicating strong discriminative capability. These results collectively validate that the proposed approach not only improves accuracy but also enhances interpretability and consistency in narrative evaluation.

Another important observation is the scalability of the system in handling large and diverse datasets. The model demonstrates stable performance across different narrative types and structures, indicating its adaptability to real-world scenarios. Unlike traditional approaches that are limited to sentence-level analysis, the proposed system effectively models document-level dependencies, enabling it to capture long-range relationships and complex narrative patterns. This capability is particularly important in detecting inconsistencies that emerge across multiple segments of a story.

Overall, the results confirm that the proposed ChronoMind framework provides a comprehensive and reliable solution for narrative coherence evaluation. By combining transformer-based learning with symbolic reasoning and tone analysis, the system achieves improved performance while maintaining interpretability, making it suitable for applications in content analysis, writing assistance, and intelligent text evaluation.

4.7 Summary

The ChronoMind system achieves high accuracy and reliable performance in narrative coherence classification. The results confirm that the model generalizes well and effectively distinguishes between coherent and incoherent narratives. The combination of neural, symbolic, and tone-based analysis provides a comprehensive and interpretable framework for narrative evaluation, demonstrating the effectiveness of the proposed approach.

In addition to its strong classification performance, the system demonstrates stability and consistency across different evaluation conditions, as evidenced by the alignment between training and validation metrics. The low error rates observed in the confusion matrix and the high ROC-AUC value further reinforce the robustness of the model in distinguishing between narrative classes. The integration of multiple analytical components ensures that the system not only captures contextual relationships but also validates logical sequencing and stylistic consistency, resulting in a more holistic evaluation of narratives.

Furthermore, the proposed framework addresses key limitations identified in existing approaches by moving beyond sentence-level analysis and incorporating document-level reasoning. This enables the system to effectively handle long-range dependencies and complex narrative structures. Overall, the findings highlight that ChronoMind provides a scalable, accurate, and interpretable solution for automated narrative coherence evaluation, making it suitable for real-world applications in content analysis, education, and intelligent writing support systems.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

The ChronoMind neuro-symbolic framework has emerged that is being used to assess logical and temporal consistency in short narratives. This framework employs a fine-tuned RoBERTa-style transformer model to capture relationships between different segments of narratives and classify these segments as coherent or incoherent. Traditional Natural Language Processing techniques are typically applied at the level of individual sentences; however, the chronological mind framework operates at the level of documents, enabling it to identify inconsistencies due to the complex relationships between events, characters, and timelines.

The use of symbolic reasoning facilitates the validation of logical relationships and chronological ordering of events, thus ensuring that the evaluation of elements within the narrative provides interpretability and an organized method to confirm whether or not the elements of the narrative establish coherence.

The tone analysis component of the system provides context with regard to stylistic features such as neutrality, irony, and sarcasm, thereby improving the understanding of narrative flow. Through a weighted fusion mechanism that integrates these various components, the ChronoMind framework for evaluating coherence is very robust and reliable.

The results of the experiments provide strong evidence to support the proposal; the results show an accuracy rate of 97.41%, an F1 score of 0.9745, and an ROC-AUC of 0.994. The very low number of false positives and false negatives supports the conclusion that the evaluation framework has performed well in classifying narratives and generalizing the results across exercises and participants. Both the training and validation analyses demonstrate consistent learning patterns and very little overfitting.

In addition to its strong quantitative performance, the ChronoMind framework demonstrates the effectiveness of integrating multiple analytical paradigms into a unified system for narrative evaluation. The combination of deep contextual understanding through transformer models, structured validation through symbolic reasoning, and stylistic interpretation through

tone analysis enables the system to address the multifaceted nature of narrative coherence. This hybrid approach not only improves classification accuracy but also enhances interpretability, allowing users to understand the reasoning behind the system's predictions.

Furthermore, the ability of the system to operate at the document level represents a significant advancement over conventional approaches, as it enables the detection of long-range dependencies and complex narrative relationships that are often overlooked in sentence-level analysis. The framework's consistent performance across training and validation phases highlights its robustness and generalization capability, making it suitable for handling diverse narrative structures and real-world datasets.

Another important contribution of this work lies in its practical applicability. The system can be extended to support a wide range of applications, including automated content evaluation, intelligent writing assistance, and educational tools for improving narrative construction. By providing both structural and contextual insights, ChronoMind offers a comprehensive solution for analyzing narrative quality in a scalable and efficient manner.

Overall, the proposed framework successfully achieves its objective of developing an intelligent, interpretable, and high-performing system for narrative coherence evaluation. The results validate the effectiveness of the approach and establish a strong foundation for future research in advanced narrative understanding, hybrid AI systems, and intelligent content analysis.

5.2 Future Enhancements

ChronoMind has achieved great performance, but there are still many ways in which ChronoMind could be improved upon and developed. For example, currently, ChronoMind could be enhanced to allow for processing of longer and more complex narrative types (i.e., full-length novels

and multiple documents) that are more in-depth in context. Improvements could also include optimizing the model for larger context windows and enhancing how quickly it computes information.

In another area of opportunity, developing more effective advanced temporal reasoning techniques (i.e., graph model constructions of events and event causal relationship tracking)

would enhance the system's ability to account for complex narrative dependencies more effectively. Furthermore, the system could enhance its symbolic reasoning component by integrating deeper formal logic solvers, allowing users to be more accurate in their determination of contradictions and logical inconsistencies within the narrative. Additionally, the tone analysis portion of the system could be improved by adding in more advanced emotion patterns and stylistic features (e.g., sentiment intensity and character mood transitions). Integrating multi-modal (e.g., text combined with audio and/or video) analyses into the system would also increase the amount of domains to which the system could be used. From an application standpoint, the system could be used as an intelligent writing assistant, learning tool, or moderation program. In real-time, users can receive feedback to help them improve on the quality of their narratives while writing.

Beyond these improvements, future work can also focus on enhancing the scalability and deployment capabilities of the ChronoMind system. Integrating the framework into cloud-based infrastructures would allow it to handle large-scale data processing and support real-time applications across multiple users. Additionally, optimizing the model for deployment on edge devices or low-resource environments could expand its accessibility and usability in practical scenarios. This includes exploring model compression techniques, such as knowledge distillation and parameter pruning, to reduce computational overhead while maintaining performance.

Another promising direction is the incorporation of explainable artificial intelligence (XAI) techniques to improve the transparency of model predictions. Providing interpretable explanations for coherence decisions—such as highlighting inconsistent segments or explaining logical conflicts—would significantly enhance user trust and usability, especially in educational and research contexts. This would allow users not only to receive a classification result but also to understand the reasoning behind it.

Furthermore, the system can be extended to support multilingual narrative analysis, enabling evaluation across different languages and cultural contexts. This would require adapting the model to handle diverse linguistic structures and training it on multilingual datasets. Expanding the dataset to include more diverse and domain-specific narratives, such as legal documents, historical texts, or conversational storytelling, could also improve the robustness and adaptability of the system.

Finally, future enhancements could explore the integration of interactive and adaptive learning mechanisms, where the system continuously improves based on user feedback. By incorporating reinforcement learning or feedback-driven optimization, ChronoMind could evolve into a dynamic system capable of refining its predictions over time. These advancements would further strengthen the system's capability to serve as a comprehensive, intelligent platform for narrative analysis and content evaluation.

5.3 Final Remarks

ChronoMind advances intelligent understanding of narratives significantly. The platform utilizes top features of transformer-based models, symbolic reasoning, and tone analysis together as one whole. As a result, ChronoMind functions at extremely high levels and provides clear, interpretable, and structured methods for assessing the coherence of narrative. By incorporating multiple analysis methods to get rid of limitations inherent in current sentence level narrative coherence assessments, ChronoMind will help automate text analysis and assess intelligent content. ChronoMind's research results indicate that it is possible to combine different types of analytical approaches to solve challenging problems associated with understanding language.

In addition to its technical contributions, ChronoMind highlights the importance of adopting interdisciplinary approaches in solving complex natural language understanding problems. By bridging the gap between data-driven learning and rule-based reasoning, the framework demonstrates how hybrid systems can provide both high performance and interpretability. This balance is particularly important in applications where transparency and reliability are essential, such as education, content evaluation, and research analysis.

Moreover, the system establishes a foundation for future advancements in narrative intelligence by enabling deeper exploration of document-level understanding, contextual reasoning, and stylistic interpretation. The ability to analyze narratives beyond surface-level semantics opens new possibilities for developing intelligent systems that can assist in creative writing, automated content generation, and critical text analysis. ChronoMind thus contributes not only as a standalone solution but also as a stepping stone for further innovation in artificial intelligence and natural language processing.

Overall, the work emphasizes that effective narrative evaluation requires a comprehensive perspective that integrates structure, context, logic, and style. The successful implementation of ChronoMind reinforces the potential of hybrid AI systems in addressing real-world challenges and advancing the capabilities of intelligent content understanding systems.

REFERENCES

[1] "CONTRADOC: Understanding self-contradictions in documents with large language models," in *Proc. 2024 Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2024.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS),* 2017.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[4] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[5] "Narrative-of-thought: Improving temporal reasoning of large language models via recounted narratives," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024.

[6] "Learning to reason over time: Timeline self-reflection for improved temporal reasoning in language models," *arXiv preprint*, 2025.

[7] "NeSTR: A neuro-symbolic abductive framework for temporal reasoning in large language models," *arXiv preprint*, 2025.

[8] "SCORE: Story coherence and retrieval enhancement for AI narratives," *arXiv preprint*, 2025.

[9] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[10] "SemEval shared tasks on irony and sarcasm detection," in *Proc. Int. Workshop on Semantic Evaluation (SemEval),* 2018.

APPNDIX
A CODING

```
app.py — ChronoMind Narrative Analyser
Streamlit app with:
  • Login page
  • Page 1: PDF Upload → Coherence + Tone analysis
  • Page 2: Analytics Dashboard (training plots + logs)
"""
import io
import os
import re
import json
import datetime
import textwrap
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
import streamlit as st
try:
    import pdfplumber
    HAS_PDFPLUMBER = True
except ImportError:
    HAS_PDFPLUMBER = False
try:
    import PyPDF2
    HAS_PYPDF2 = True
except ImportError:
    HAS_PYPDF2 = False
```

```
from tone_analyzer import ToneAnalyzer
from coherence_analyzer import CoherenceAnalyzer
from auth_db import init_db, verify_user, add_user

# Initialise SQLite auth database (creates file + seeds defaults on first run)
init_db()

# ── Constants ──────────────────────────────────────────────────────────────

PLOTS_DIR = os.path.join(os.path.dirname(__file__), "plots")

PLOT_CATALOG = {
    "  Training Curves":  "training_curves.png",
}

LOG_FILE = os.path.join(os.path.dirname(__file__), "analysis_log.json")

# ── Page config ────────────────────────────────────────────────────────────

st.set_page_config(
    page_title="ChronoMind",
    page_icon="  ",
    layout="wide",
    initial_sidebar_state="expanded",
)
# ── Custom CSS ─────────────────────────────────────────────────────────────

st.markdown("""
<style>
@import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;70
0&display=swap');

html, body, [class*="css"] { font-family: 'Inter', sans-serif; }
```

```
/* ── sidebar */
section[data-testid="stSidebar"]  {
    background: linear-gradient(160deg, #0f0c29, #302b63, #24243e);
    color: #e2e8f0;
}
section[data-testid="stSidebar"] * { color: #e2e8f0 !important; }
section[data-testid="stSidebar"] .stRadio > label { font-weight: 600; font-size:
0.85rem; letter-spacing: 0.05em; }

/* ── main background */
.main { background: #0d1117; color: #c9d1d9; }
.block-container { padding-top: 2rem; }

/* ── cards */
.card {
    background: rgba(255,255,255,0.04);
    border: 1px solid rgba(255,255,255,0.08);
    border-radius: 16px;
    padding: 1.5rem 1.8rem;
    margin-bottom: 1.2rem;
    backdrop-filter: blur(8px);
}
.card h3 { margin: 0 0 0.6rem; font-size: 1rem; font-weight: 600; letter-spacing:
0.03em; color: #a0aec0; }
.card .value { font-size: 2rem; font-weight: 700; }

/* ── verdict badges */
.badge-coherent
    { display: inline-
    block;
    background: linear-gradient(135deg, #065f46, #059669);
    color: #ecfdf5;
    font-size: 1.5rem; font-weight: 700;
    padding: 0.5rem 1.8rem;
    border-radius: 999px;
```

```
    letter-spacing: 0.04em;
    box-shadow: 0 4px 20px rgba(5,150,105,0.4);
}
.badge-incoherent
    { display: inline-
    block;
    background: linear-gradient(135deg, #7f1d1d, #dc2626);
    color: #fef2f2;
    font-size: 1.5rem; font-weight: 700;
    padding: 0.5rem 1.8rem;
    border-radius: 999px;
    letter-spacing: 0.04em;
    box-shadow: 0 4px 20px rgba(220,38,38,0.4);
}
/* ── tone pill */
.tone-pill {
    display: inline-block;
    background: linear-gradient(135deg, #1e3a5f, #2563eb);
    color: #dbeafe;
    font-size: 1rem; font-weight: 600;
    padding: 0.4rem 1.4rem;
    border-radius: 999px;
    margin-top: 0.4rem;
}

/* ── section headers */
.section-title {
    font-size: 1.25rem; font-weight: 700;
    color: #fff;
    border-left: 3px solid #6366f1;
    padding-left: 0.75rem; margin:
    1.5rem 0 0.8rem;
}
/* ── dashboard title */
.dash-title {
```

```
    font-size: 2.2rem; font-weight: 800;
    background: linear-gradient(90deg, #a78bfa, #60a5fa, #34d399);
    -webkit-background-clip: text; -webkit-text-fill-color: transparent;
    margin-bottom: 0.2rem;
}

/* ── metric mini-card */
.mini-card {
    background: rgba(255,255,255,0.05);
    border: 1px solid rgba(255,255,255,0.09);
    border-radius: 12px;
    padding: 1rem;
    text-align: center;
}
.mini-label { font-size: 0.75rem; color: #94a3b8; font-weight: 500; text-
transform: uppercase; letter-spacing: 0.06em; }
.mini-value { font-size: 1.6rem; font-weight: 700; color: #f1f5f9; margin-top:
0.2rem; }

/* Plotly charts dark bg */
.js-plotly-plot { border-radius: 12px; overflow: hidden; }

/* log table */
.log-row { font-size: 0.82rem; border-bottom: 1px solid rgba(255,255,255,0.06);
padding: 0.5rem 0; }
/* ── login page */
.login-box {
    max-width: 420px;
    margin: 6rem auto 0;
    background: rgba(255,255,255,0.04);
    border: 1px solid rgba(255,255,255,0.09);
    border-radius: 20px;
    padding: 2.5rem 2.8rem;
    backdrop-filter: blur(10px);
    box-shadow: 0 8px 40px rgba(0,0,0,0.5);
```

```
}
.login-title {
    font-size: 2rem; font-weight: 800;
    background: linear-gradient(90deg, #a78bfa, #60a5fa);
    -webkit-background-clip: text; -webkit-text-fill-color: transparent;
    text-align: center; margin-bottom: 0.3rem;
}
.login-sub {
    text-align: center; color: #64748b; font-size: 0.85rem; margin-bottom:
1.6rem;
}
</style>
""", unsafe_allow_html=True)

# ── Login gate ──────────────────────────────────────────────────────────

if "authenticated" not in st.session_state:
    st.session_state.authenticated = False

if not st.session_state.authenticated:
    # Centre the auth card
    _, mid, _ = st.columns([1, 1.4, 1])
    with mid:
        st.markdown('<div class="login-title">  ChronoMind</div>',
unsafe_allow_html=True)
        st.markdown("<br>", unsafe_allow_html=True)

        tab_login, tab_register = st.tabs(["  Sign In  ", "  Register  "])

        # ── LOGIN TAB ──────────────────────────────────────────────────
        with tab_login:
            st.markdown("<br>", unsafe_allow_html=True)
            uname = st.text_input("Username", placeholder="Enter username",
key="li_user")
```

```
            pwd     = st.text_input("Password", type="password",
placeholder="Enter password", key="li_pwd")

            st.markdown("<br>", unsafe_allow_html=True)

            login_btn = st.button("      Sign In", use_container_width=True,
key="btn_login")


            if login_btn:

                if not uname or not pwd:

                    st.warning("     Please fill in both fields.")

                else:

                    user = verify_user(uname, pwd)

                    if user:

                        st.session_state.authenticated = True

                        st.session_state.username          = user["username"]

                        st.session_state.role              = user["role"]

                        st.rerun()

                    else:

                        st.error("✕ Invalid username or password.")


        # ── REGISTER TAB
──────────────────────────────────────────────────────

        with tab_register:

            st.markdown("<br>", unsafe_allow_html=True)

            reg_user  = st.text_input("Choose a username", placeholder="e.g.
john_doe", key="reg_user")

            reg_pwd     = st.text_input("Choose a password", type="password",

                                        placeholder="Min 6 characters",
key="reg_pwd")

            reg_pwd2  = st.text_input("Confirm password", type="password",

                                        placeholder="Repeat password",
key="reg_pwd2")

            st.markdown("<br>", unsafe_allow_html=True)

            reg_btn = st.button("      Create Account", use_container_width=True,
key="btn_register")


            if reg_btn:
```

```
                # Validation
                if not reg_user or not reg_pwd or not reg_pwd2:
                    st.warning("    All fields are required.")
                elif len(reg_user.strip()) < 3:
                    st.warning("    Username must be at least 3 characters.")
                elif len(reg_pwd) < 6:
                    st.warning("    Password must be at least 6 characters.")
                elif reg_pwd != reg_pwd2:
                    st.error("❌ Passwords do not match.")
                else:
                    ok = add_user(reg_user.strip(), reg_pwd, role="user")
                    if ok:
                        st.success(f"✅ Account created for **{reg_user}**!
Switch to Sign In to log in.")
                    else:
                        st.error("❌ Username already taken. Please choose a
different one.")

        st.markdown("<br>", unsafe_allow_html=True)
        st.markdown("""
        <div style='text-align:center; font-size:0.75rem; color:#475569;'>
          Made by <b style='color:#a78bfa;'>Sreenadh</b> (RA2211003011436)
&amp;amp;
          <b style='color:#60a5fa;'>Jaswanth</b> (RA2211003011414)
        </div>
        """, unsafe_allow_html=True)
    st.stop()

# ── Helpers ──────────────────────────────────────────────────────────────
──────────────────────────────

@st.cache_resource
def get_analyzers():
    return ToneAnalyzer(), CoherenceAnalyzer()
```

```
tone_analyzer, coherence_analyzer = get_analyzers()

def extract_text(uploaded_file) -> str:
    """Extract text from PDF using pdfplumber (preferred) or PyPDF2."""
    raw = uploaded_file.read()
    text = ""

    if HAS_PDFPLUMBER:
        with pdfplumber.open(io.BytesIO(raw)) as pdf:
            for page in pdf.pages:
                t = page.extract_text()
                if t:
                    text += t + "\n"
    if not text.strip() and HAS_PYPDF2:
        reader = PyPDF2.PdfReader(io.BytesIO(raw))
        for page in reader.pages:
            t = page.extract_text()
            if t:
                text += t + "\n"
    return text.strip()

def split_sentences(text: str) -> list[str]:
    parts = re.split(r'(?<=[.!?])\s+', text)
    return [p.strip() for p in parts if p.strip() and len(p.split()) >= 3]

def load_logs() -> list[dict]:
    if os.path.exists(LOG_FILE):
        try:
            with open(LOG_FILE, "r") as f:
                return json.load(f)
        except Exception:
            pass
    return []
```

```
def save_log(entry: dict):
    logs = load_logs()
    logs.append(entry)
    with open(LOG_FILE, "w") as f:
        json.dump(logs[-200:], f, indent=2)       # keep last 200


def dark_plotly(fig: go.Figure) -> go.Figure:
    fig.update_layout(
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(255,255,255,0.03)",
        font_color="#c9d1d9",
        xaxis=dict(gridcolor="rgba(255,255,255,0.06)", showline=False),
        yaxis=dict(gridcolor="rgba(255,255,255,0.06)", showline=False),
        margin=dict(l=20, r=20, t=40, b=20),
    )
    return fig


# ── Sidebar navigation ──────────────────────────────────────────────────────

st.sidebar.markdown("""
<div style='text-align:center; padding: 1.2rem 0 0.5rem;'>
  <span style='font-size:2.5rem;'>  </span><br>
  <span style='font-size:1.25rem; font-weight:800; letter-spacing:0.04em;'>ChronoMind</span>
</div>
<hr style='border:1px solid rgba(255,255,255,0.1); margin: 0.8rem 0;'>
""", unsafe_allow_html=True)


page = st.sidebar.radio( "Naviga tion", ["    PDF Analyser", "    Analytics Dashboard"], label_visibility="collapsed",
)
```

```
st.sidebar.markdown("<hr style='border:1px solid rgba(255,255,255,0.1);'>",
unsafe_allow_html=True)

st.sidebar.markdown("""

<div style='font-size:0.75rem; color:#94a3b8; padding: 0.5rem 0; line-
height:1.6;'>

  Made by<br>

  <b style='color:#a78bfa;'>Sreenadh</b> <span
style='color:#64748b;'>(RA2211003011436)</span><br>

  <b style='color:#60a5fa;'>Jaswanth</b> <span
style='color:#64748b;'>(RA2211003011414)</span>

</div>

""", unsafe_allow_html=True)


# Logout button

if st.sidebar.button("     Logout", use_container_width=True):

    st.session_state.authenticated = False

    st.rerun()


#
═══════════════════════════════════════════════════════════
═══════════════════════════════════

#  PAGE 1 — PDF ANALYSER

#
═══════════════════════════════════════════════════════════
═══════════════════════════════════

if page == "     PDF Analyser":


    st.markdown('<div class="dash-title">     PDF Analyser</div>',
unsafe_allow_html=True)

    st.markdown('<p style="color:#64748b; margin-bottom:1.5rem;">Upload a
PDF to instantly detect narrative coherence and emotional tone.</p>',
unsafe_allow_html=True)


    uploaded = st.file_uploader("Drop your PDF here", type=["pdf"],
label_visibility="collapsed")


    if uploaded is None:

        st.markdown("""
```

```
        <div style='border: 2px dashed rgba(255,255,255,0.1); border-
radius:16px;
                    padding: 3rem; text-align:center; color:#475569; margin-
top:1rem;'>
          <div style='font-size:3rem; margin-bottom:0.5rem;'>  </div>
          <div style='font-size:1.1rem; font-weight:500;'>Drag & drop a PDF
above</div>
          <div style='font-size:0.85rem; margin-top:0.4rem;'>Supports narrative
texts, reports, stories</div>
        </div>
        """, unsafe_allow_html=True)


    else:
        with st.spinner("    Extracting & analysing text…"):
            raw_text = extract_text(uploaded)


        if not raw_text or len(raw_text.split()) < 20:
            st.error("    Could not extract enough text from this PDF. Try a
different file.")
        else:
            sentences  = split_sentences(raw_text)
            coh_result = coherence_analyzer.analyze(raw_text)
            tone_result = tone_analyzer.analyze(sentences)


            # ── Save log
──────────────────────────────────────────────────────────────
─────
            log_entry = {
                "timestamp":    datetime.datetime.now().isoformat(timespec="se
conds"),
                "filename":     uploaded.name,
                "verdict":      coh_result["verdict"],
                "score":        coh_result["score"],
                "tone_label": tone_result["tone_label"],
                "dominant":     tone_result["dominant_emotion"],
                "word_count":  coh_result["details"].get("word_count", 0),
            }
```

```
        save_log(log_entry)

        #
═══════════════════════════════════════════════════
        #  ROW 1 — Verdict + Tone headline
        #
═══════════════════════════════════════════════════
        st.markdown('<div class="section-title">Analysis Results</div>',
unsafe_allow_html=True)

        col_v, col_t = st.columns([1, 1], gap="large")

        with col_v:
            badge_cls = "badge-coherent" if coh_result["coherent"] else
"badge-incoherent"
            icon      = "✓" if coh_result["coherent"] else "✗"
            st.markdown(f"""
            <div class="card" style="text-align:center;">
              <h3>Narrative Coherence</h3>
              <div style="margin: 0.8rem 0;">
                <span class="{badge_cls}">{icon}
{coh_result['verdict']}</span>
              </div>
              <div style="color:#94a3b8; font-size:0.85rem; margin-
top:0.8rem;">
                Confidence: <b
style="color:#f1f5f9">{coh_result['confidence']:.0%}</b>
              </div>
            </div>
            """, unsafe_allow_html=True)

            # Coherence score bar
            score_fig =
                go.Figure(go.Indicator( mode =
                "gauge+number",
                value = coh_result["score"] * 100,
                title = {"text": "Coherence Score", "font": {"size": 13}},
```

```
                    gauge = {
                        "axis":   {"range": [0, 100], "tickfont": {"size": 10}},
                        "bar":     {"color": "#059669" if coh_result["coherent"]
else "#dc2626"},
                        "bgcolor": "rgba(0,0,0,0)",
                        "steps": [
                            {"range": [0, 55],   "color": "rgba(220,38,38,0.15)"},
                            {"range": [55, 100], "color": "rgba(5,150,105,0.15)"},
                        ],
                        "threshold": {"line": {"color": "white", "width": 2},
"value": 55},
                    },
                    number={"suffix": "%", "font": {"size": 22}},
                ))
                dark_plotly(score_fig)
                score_fig.update_layout(height=220)
                st.plotly_chart(score_fig, use_container_width=True)


            with col_t:
                trend_arrow = {"improving": "   ", "declining": "   ", "stable":
"  "}.get(tone_result["trend"], "  ")
                dom_color     = {"positive": "#059669", "negative": "#dc2626",
"neutral": "#94a3b8"}
                dom_c         = dom_color.get(tone_result["dominant_emotion"],
"#94a3b8")


                st.markdown(f"""
                <div class="card" style="text-align:center;">
                  <h3>Emotional Tone</h3>
                  <div style="margin:0.6rem 0;">
                    <span class="tone-
pill">{tone_result['tone_emoji']}  {tone_result['tone_label']}</span>
                </div>
                <div style="margin-top:0.9rem; display:flex; justify-
content:space-around;">
                  <div>
```

```
                    <div class="mini-label">Dominant</div>
                    <div style="font-size:1.1rem; font-weight:700;
color:{dom_c};">{tone_result['dominant_emotion'].capitalize()}</div>
                  </div>
                  <div>
                    <div class="mini-label">Trend</div>
                    <div style="font-size:1.1rem; font-weight:700;
color:#f1f5f9;">{trend_arrow} {tone_result['trend'].capitalize()}</div>
                  </div>
                  <div>
                    <div class="mini-label">Consistency</div>
                    <div style="font-size:1.1rem; font-weight:700;
color:#f1f5f9;">{tone_result['consistency']:.0%}</div>
                  </div>
                </div>
              </div>
              """, unsafe_allow_html=True)

              # Sentiment breakdown donut
              donut_fig = go.Figure(go.Pie(
                  labels=["Positive", "Neutral", "Negative"],
                  values=[
                      tone_result["pos_ratio"],
                      tone_result["neu_ratio"],
                      tone_result["neg_ratio"],
                  ],
                  hole=0.62,
                  marker_colors=["#059669", "#64748b", "#dc2626"],
                  textfont={"size": 12},
                  hovertemplate="%{label}: %{percent}<extra></extra>",
              ))
              donut_fig.update_layout( showlegend=Tru
                  e,
                  legend=dict(orientation="h", y=-0.1, font=dict(size=11)),
                  height=220,
```

```
            margin=dict(l=10, r=10, t=10, b=10),
            paper_bgcolor="rgba(0,0,0,0)",
            font_color="#c9d1d9",
        )
        st.plotly_chart(donut_fig, use_container_width=True)

    #
═══════════════════════════════════════════════════
    #  ROW 2 — Emotional Arc
    #
═══════════════════════════════════════════════════
    if tone_result["sentence_scores"]:
        st.markdown('<div class="section-title">     Emotional
Arc</div>', unsafe_allow_html=True)

        arc_scores = tone_result["sentence_scores"]
        x_vals       = list(range(1, len(arc_scores) + 1))

        arc_fig = go.Figure()
        # Fill area
        arc_fig.add_trace(go.Scatter( x=x_val
            s, y=arc_scores, fill="tozeroy",
            fillcolor="rgba(99,102,241,0.12)",
            line=dict(color="#818cf8", width=2.5),
            mode="lines",
            name="Sentiment",
            hovertemplate="Sentence %{x}<br>Score: %{y:.3f}<extra>

</extra>",
        ))
        # Zero line
        arc_fig.add_hline(y=0, line_dash="dash",
line_color="rgba(255,255,255,0.2)", line_width=1)
        # Positive/Negative thresholds
        arc_fig.add_hline(y=0.2,  line_dash="dot",
line_color="rgba(5,150,105,0.5)", line_width=1, annotation_text="+ve zone",
```

```
annotation_font_color="#059669")
            arc_fig.add_hline(y=-0.2, line_dash="dot",
line_color="rgba(220,38,38,0.5)", line_width=1, annotation_text="-ve zone",
annotation_font_color="#dc2626")

            arc_fig.update_layout( xaxis_title
                ="Sentence #",
                yaxis_title="VADER Score",
                yaxis_range=[-1.05, 1.05],
                height=320,
            )
            dark_plotly(arc_fig)
            st.plotly_chart(arc_fig, use_container_width=True)

        #
═══════════════════════════════════════════════════
        #  ROW 3 — Mini metrics + details
        #
═══════════════════════════════════════════════════
        st.markdown('<div class="section-title"> Text Metrics</div>',
unsafe_allow_html=True)
        d = coh_result["details"]
        m_cols = st.columns(5)
        metrics = [
            ("Words",          d.get("word_count", "—")),
            ("Sentences",      d.get("sentence_count", "—")),
            ("Vocab (TTR)", f"{d.get('ttr', 0):.2f}"),
            ("Avg Score",      f"{tone_result['avg_score']:+.3f}"),
            ("Swings",         d.get("sentiment_swings", "—")),
        ]
        for col, (lbl, val) in zip(m_cols, metrics):
            with col:
                st.markdown(f"""
                <div class="mini-card">
                  <div class="mini-label">{lbl}</div>
                  <div class="mini-value">{val}</div>
```

```
                </div>
                """, unsafe_allow_html=True)

            # ── Reasons
            st.markdown('<div class="section-title">  Coherence Signals</div>', unsafe_allow_html=True)
            for r in coh_result["reasons"]:
                st.markdown(f"<div style='padding:0.3rem 0; color:#cbd5e1; font-size:0.9rem;'>{r}</div>", unsafe_allow_html=True)

            # ── Raw text preview
            with st.expander("  Extracted Text Preview"):
                st.text(textwrap.fill(raw_text[:3000], 90) + ("…" if len(raw_text) > 3000 else ""))

#
═══════════════════════════════════════════════════════════════════════
# PAGE 2 — ANALYTICS DASHBOARD
#
═══════════════════════════════════════════════════════════════════════
elif page == "  Analytics Dashboard":

    st.markdown('<div class="dash-title">  Analytics Dashboard</div>', unsafe_allow_html=True)
    st.markdown('<p style="color:#64748b; margin-bottom:1.8rem;">Model evaluation plots and session analysis log.</p>', unsafe_allow_html=True)

    # ── Plot viewer
    ──────────────────────────────────────────────────────────────────────

    st.markdown('<div class="section-title">  Model & Evaluation Plots</div>', unsafe_allow_html=True)

    plot_choice =
        st.selectbox( "Select a
        plot",
        list(PLOT_CATALOG.keys()),
```

```
        index=0,
    )

    plot_path = os.path.join(PLOTS_DIR, PLOT_CATALOG[plot_choice])
    if os.path.exists(plot_path):
        st.markdown(f"<div style='text-align:center; margin-top:0.5rem;'>",
unsafe_allow_html=True)
        st.image(plot_path, use_container_width=True, caption=plot_choice)
        st.markdown("</div>", unsafe_allow_html=True)
    else:
        st.warning(f"Plot file not found: {PLOT_CATALOG[plot_choice]}")

    # Thumbnail strip omitted — only one plot in catalog

    # ── Session Log
──────────────────────────────────────────────────────────────────
──────────

    st.markdown("---")
    st.markdown('<div class="section-title">  Analysis Session Log</div>',
unsafe_allow_html=True)

    logs = load_logs()
    if not logs:
        st.info("No analyses yet. Upload a PDF on the Analyser page to
populate the log.")
    else:
        logs_rev = list(reversed(logs))  # newest first

        # Summary KPIs
        k1, k2, k3, k4 = st.columns(4)
        total     = len(logs)
        coherent  = sum(1 for l in logs if l["verdict"] == "Coherent")
        avg_score = np.mean([l["score"] for l in logs])
        pos_tone  = sum(1 for l in logs if l["dominant"] == "positive")

        for col, lbl, val in [
```

```
            (k1, "Total Analyses",        total),
            (k2, "Coherent Docs",         f"{coherent}/{total}"),
            (k3, "Avg Coh. Score",        f"{avg_score:.0%}"),
            (k4, "Positive Tone Docs", pos_tone),
        ]:
            with col:
                st.markdown(f"""
                <div class="mini-card" style="margin-bottom:1rem;">
                  <div class="mini-label">{lbl}</div>
                  <div class="mini-value">{val}</div>
                </div>
                """, unsafe_allow_html=True)

        # Coherence trend line
        if len(logs) >= 2:
            df_log = pd.DataFrame(logs)
            df_log["idx"] = range(1, len(df_log) + 1)
            trend_fig = px.line(
                df_log, x="idx", y="score",
                labels={"idx": "Analysis #", "score": "Coherence Score"},
                title="Coherence Score Over Sessions",
                markers=True,
            )
            trend_fig.update_traces(line_color="#6366f1",
marker_color="#818cf8")
            dark_plotly(trend_fig)
            trend_fig.update_layout(height=260, title_font_size=14)
            st.plotly_chart(trend_fig, use_container_width=True)

        # Log table
        df_display = pd.DataFrame(logs_rev)[
            ["timestamp", "filename", "verdict", "score", "tone_label",
"dominant", "word_count"]
        ].rename(columns={
```

```
        "timestamp":  "Time",
        "filename":     "File",
        "verdict":      "Verdict",
        "score":        "Score",
        "tone_label": "Tone",
        "dominant":     "Emotion",
        "word_count": "Words",
    })
    df_display["Score"] = df_display["Score"].map(lambda x: f"{x:.0%}")

    st.dataframe( df_disp
        lay,
        use_container_width=True,
        hide_index=True,
        column_config={
            "Verdict": st.column_config.TextColumn(width="small"),
            "Tone":      st.column_config.TextColumn(width="medium"),
        }
    )

    col_dl, _ = st.columns([1, 3])
    with col_dl:
        csv = df_display.to_csv(index=False).encode("utf-8")
        st.download_button("      Download Log CSV", csv,

        "analysis_log.csv", "text/csv")
```

auth_db.py — SQLite authentication layer for ChronoMind.

```
Tables
    -------
    users(id, username, password_hash, role, created_at)
    Passwords are stored as SHA-256 hashes (hex digest).
    """
    import sqlite3
    import hashlib
    import os
    import datetime
    DB_PATH = os.path.join(os.path.dirname(__file__),
    "chronmind_auth.db")
    # ── Internal helpers
    ──────────────────────────────────────────────────────────────
    ──────────────────────────
    def _hash(password: str) -> str:
        """Return SHA-256 hex digest of the password."""
        return hashlib.sha256(password.encode()).hexdigest()
    def _connect() -> sqlite3.Connection:
        conn = sqlite3.connect(DB_PATH, check_same_thread=False)
        conn.row_factory = sqlite3.Row
        return conn
    # ── Schema init
    ──────────────────────────────────────────────────────────────
    ──────────────────────────────
    def init_db():
        """Create tables and seed default users if they don't exist."""
        with _connect() as conn:
```

```
    conn.execute("""
        CREATE TABLE IF NOT EXISTS users
            ( id            INTEGER PRIMARY KEY
AUTOINCREMENT,
            username        TEXT      NOT NULL UNIQUE
COLLATE NOCASE,
            password_hash TEXT      NOT NULL,
            role            TEXT      NOT NULL DEFAULT 'user',
            created_at      TEXT      NOT NULL
        )
    """)
    conn.commit()
    # Seed default accounts (only if table is empty)
    cur = conn.execute("SELECT COUNT(*) FROM users")
    if cur.fetchone()[0] == 0:
        now =
datetime.datetime.now().isoformat(timespec="seconds")
        defaults = [
            ("admin",       "admin123",     "admin"),
            ("sreenadh",    "chrono2024", "user"),
            ("jaswanth",    "chrono2024", "user"),
        ]
        conn.executemany(
            "INSERT INTO users (username, password_hash, role,
created_at) VALUES (?,?,?,?)",
            [(u, _hash(p), r, now) for u, p, r in defaults],
        )
```

```
            conn.commit()

# ── Public API ──────────────────────────────────────────────────────────

def verify_user(username: str, password: str) -> dict | None:
    """
    Return user row dict if credentials are valid, else None.
    Keys: id, username, role, created_at
    """
    with _connect() as conn:
        cur = conn.execute(
            "SELECT * FROM users WHERE username = ? AND
password_hash = ?",
            (username.strip(), _hash(password)),
        )
        row = cur.fetchone()
        return dict(row) if row else None


def add_user(username: str, password: str, role: str = "user") -> bool:
    """
    Add a new user. Returns True on success, False if username already
exists.
    """
    try:
        now = datetime.datetime.now().isoformat(timespec="seconds")
        with _connect() as conn:
            conn.execute(
```

```
                "INSERT INTO users (username, password_hash, role,
created_at) VALUES (?,?,?,?)",
                (username.strip(), _hash(password), role, now),
            )
            conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False


def list_users() -> list[dict]:
    """Return all users (without password hashes)."""
    with _connect() as conn:
        cur = conn.execute("SELECT id, username, role, created_at
FROM users ORDER BY id")
        return [dict(r) for r in cur.fetchall()]
def delete_user(username: str) -> bool:
    """Delete a user by username. Returns True if a row was deleted."""
    with _connect() as conn:
        cur = conn.execute("DELETE FROM users WHERE username = ?
COLLATE NOCASE", (username,))
        conn.commit()
        return cur.rowcount > 0
def change_password(username: str, new_password: str) -> bool:
    """Update a user's password. Returns True if row was updated."""
    with _connect() as conn:
        cur = conn.execute(
```

```
            "UPDATE users SET password_hash = ? WHERE username
= ? COLLATE NOCASE",
            (_hash(new_password), username),
        )
        conn.commit()
        return cur.rowcount > 0

"""
coherence_analyzer.py
Heuristic-based narrative coherence detection:
  - Vocabulary richness (Type-Token Ratio)
  - Sentence length variance
  - Word repetition ratio
  - VADER sentiment swing detection between adjacent sentences
"""
import re
import numpy as np
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
class CoherenceAnalyzer:
    """
    Analyses textual coherence without a heavy ML model.
    Returns a score 0→1 (1 = fully coherent) and a verdict.
    """
    SWING_THRESHOLD = 0.6     # compound diff that counts as a
sentiment swing
```

```
MIN_TTR              = 0.30  # below this = too repetitive
MAX_LEN_CV           = 2.5     # above this = erratic sentence lengths


def __init__(self):
    self._vader = SentimentIntensityAnalyzer()
# ------------------------------------------------------------------------
# helpers
# ------------------------------------------------------------------------
@staticmethod
def _tokenize(text: str) -> list[str]:
    return re.findall(r"[a-zA-Z']+", text.lower())
@staticmethod
def _split_sentences(text: str) -> list[str]:
    # Simple regex sentence splitter
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    return [s.strip() for s in sentences if s.strip()]
# ------------------------------------------------------------------------
# individual metrics
# ------------------------------------------------------------------------
 def _ttr(self, words: list[str]) -> tuple[float, str]:
    """Type-Token Ratio — measures vocabulary diversity."""
    if not words:
        return 0.5, 'insufficient text'
    ttr = len(set(words)) / len(words)
```

```
        if ttr < self.MIN_TTR:
            return ttr, f'low vocabulary diversity (TTR={ttr:.2f})'
        return ttr, 'good vocabulary diversity'
    def _length_variance(self, sentences: list[str]) -> tuple[float, str]:
        """Coefficient of variation of sentence lengths."""
        lengths = [len(s.split()) for s in sentences if s.split()]
        if len(lengths) < 2:
            return 0.0, 'too few sentences'
        mean = np.mean(lengths)
        std  = np.std(lengths)
        cv   = std / mean if mean > 0 else 0
        if cv > self.MAX_LEN_CV:
            return cv, f'highly erratic sentence lengths (CV={cv:.2f})'
        return cv, 'sentence lengths are regular'
    def _sentiment_swings(self, sentences: list[str]) -> tuple[int, str]:
        """Count large adjacent sentiment swings."""
        scores = [self._vader.polarity_scores(s)['compound'] for s in
sentences]
        swings = 0
        for i in range(1, len(scores)):
            if abs(scores[i] - scores[i - 1]) > self.SWING_THRESHOLD:
                swings += 1
        ratio = swings / max(len(sentences) - 1, 1)
        if ratio > 0.35:
```

```
            return swings, f'many abrupt sentiment swings ({swings}
detected)'
        return swings, 'sentiment flow is smooth'
    def _repetition(self, words: list[str]) -> tuple[float, str]:
        """Fraction of words that are high-frequency filler."""
        if not words:
            return 0.0, 'insufficient text'
        from collections import Counter
        counts = Counter(words)
        top_5_count = sum(v for _, v in counts.most_common(5))
        rep_ratio   = top_5_count / len(words)
        if rep_ratio > 0.50:
            return rep_ratio, f'high word repetition (top-5 words =
{rep_ratio:.0%} of text)'
        return rep_ratio, 'acceptable word variety'
    # ------------------------------------------------------------------
    # main entry
    # ------------------------------------------------------------------
    def analyze(self, text: str) -> dict:
        sentences = self._split_sentences(text)
        words     = self._tokenize(text)
        if len(sentences) < 2 or len(words) < 10:
            return {
                'coherent':     True,
```

```
            'score':           0.70,
            'confidence':  0.50,
            'verdict':         'Coherent',
            'reasons':         ['Text too short for deep analysis'],
            'details':         {},
        }

    reasons  = []
    penalties = 0.0
    # 1. TTR
    ttr, ttr_msg = self._ttr(words)
    if ttr < self.MIN_TTR:
        penalties += 0.25
        reasons.append(f'      {ttr_msg}')
    # 2. Sentence length variance
    cv, cv_msg = self._length_variance(sentences)
    if cv > self.MAX_LEN_CV:
        penalties += 0.20
        reasons.append(f'      {cv_msg}')

    # 3. Sentiment swings
    swings, sw_msg = self._sentiment_swings(sentences)
    swing_ratio = swings / max(len(sentences) - 1, 1)
    if swing_ratio > 0.35:
```

```
            penalties += 0.30
            reasons.append(f'      {sw_msg}')
        # 4. Repetition
        rep, rep_msg = self._repetition(words)
        if rep > 0.50:
            penalties += 0.15
            reasons.append(f'      {rep_msg}')
        score      = max(0.0, 1.0 - penalties)
        coherent   = score >= 0.55
        confidence = 0.50 + abs(score - 0.55) * 0.90   # confidence
grows away from boundary
        if not reasons:
            reasons.append('✓ No significant incoherence signals
detected')
        return {
            'coherent':   coherent,
            'score':      round(score, 3),
            'confidence': round(min(confidence, 0.99), 3),
            'verdict':    'Coherent' if coherent else 'Incoherent',
            'reasons':    reasons,
            'details': {
                'ttr':              round(ttr, 3),
                'len_cv':           round(cv, 3),
                'sentiment_swings': swings,
```

```
            'swing_ratio':     round(swing_ratio, 3),
            'rep_ratio':       round(rep, 3),
            'word_count':      len(words),
            'sentence_count': len(sentences),
        },
    }
```

B .PUBLICATION DETAILS

Paper ID	Paper Title	Author's Name
ICCMC-078	ChronoMind: A Neural Framework for Logical and Temporal Consistency Verification in Short Stories	Yetukuri Gargeya Sreenadh, Jaswanth Marni, Manjula Rajagopal

Dear Authors,

On behalf of the Conference committee, we would like to congratulate you on your article to the 9th International Conference on Computing Methodologies and Communication (ICCMC 2026), which will be held from **8-10, July 2026** at, **Surya Engineering College Erode,** Tamilnadu, India.

As a result of the review and results, we are pleased inform that you can now submit the camera-ready paper for inclusion into the 9th ICCMC 2026 proceedings. We appreciate if you could send the final version of your research paper at your earliest convenience, in order to ensure the timely publication. When submitting your final paper, please highlight the changes made according to the review comments.

Yours, Sincerely,

Prof. Kalaiselvi L
Program Chair
ICCMC - 2026

IEEE COPYRIGHT AND CONSENT FORM

To ensure uniformity of treatment among all contributors, other forms may not be substituted for this form, nor may any wording of the form be changed. This form is intended for original material submitted to the IEEE and must accompany any such material in order to be published by the IEEE. Please read the form carefully and keep a copy for your files.

ChronoMind: A Neural Framework for Logical and Temporal Consistency Verification in Short Stories

Yetukuri Gargeya Sreenadh, Jaswanth Marni, Manjula Rajagopal

2026 9th International Conference on Computing Methodologies and Communication (ICCMC)

COPYRIGHT TRANSFER

The undersigned hereby assigns to The Institute of Electrical and Electronics Engineers, Incorporated (the "IEEE") all rights under copyright that may exist in and to: (a) the Work, including any revised or expanded derivative works submitted to the IEEE by the undersigned based on the Work; and (b) any associated written or multimedia components or other enhancements accompanying the Work.

GENERAL TERMS

1. The undersigned represents that he/she has the power and authority to make and execute this form.
2. The undersigned agrees to indemnify and hold harmless the IEEE from any damage or expense that may arise in the event of a breach of any of the warranties set forth above.
3. The undersigned agrees that publication with IEEE is subject to the policies and procedures of the IEEE PSPB Operations Manual.
4. In the event the above work is not accepted and published by the IEEE or is withdrawn by the author(s) before acceptance by the IEEE, the foregoing copyright transfer shall be null and void. In this case, IEEE will retain a copy of the manuscript for internal administrative/record-keeping purposes.
5. For jointly authored Works, all joint authors should sign, or one of the authors should sign as authorized agent for the others.
6. The author hereby warrants that the Work and Presentation (collectively, the "Materials") are original and that he/she is the author of the Materials. To the extent the Materials incorporate text passages, figures, data or other material from the works of others, the author has obtained any necessary permissions. Where necessary, the author has obtained all third party permissions and consents to grant the license above and has provided copies of such permissions and consents to IEEE.

You have indicated that you DO wish to have video/audio recordings made of your conference presentation under terms and conditions set forth in "Consent and Release."

CONSENT AND RELEASE

1. In the event the author makes a presentation based upon the Work at a conference hosted or sponsored in whole or in part by the IEEE, the author, in consideration for his/her participation in the conference, hereby grants the IEEE the unlimited, worldwide, irrevocable

C PLAGIARISM RE

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

59 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
4% Publications
8% Submitted works (Student Papers)

turnitin Page 2 of 36 - Integrity Overview Submission ID trn:oid:::3618:137531289